

DirectAula

Datos

[dao.py](#)

```
#Archivo que contiene las clases DAO para manejar los datos
#Aquí están las bases de datos y las operaciones CRUD para cada
entidad.
#Una clase DAO (Data Access Object) se encarga de interactuar con la
base de datos para una entidad específica.
from datetime import date
import sqlite3
from model import Alumno, Asistencia, Calificacion,
CategoriaEvaluacion, Grupo
#
#La clase BASE DAO maneja la conexión a la base de datos y las
operaciones comunes.
class BaseDAO:
    def __init__(self): #Inicializa la conexión a la base de datos
SQLite
        self._db_file = 'directaula.db' #Archivo de base de datos
SQLite
        self._con = None #Conexión a la base de datos, dice none
porque aún no está conectada
        self.inicializar_tablas()

        #Aquí se asegura que las tablas necesarias existan al iniciar
el DAO
    def inicializar_tablas(self):
        #Asegura que todas las tablas necesarias existan.
        self.ejecutar_query("""
            CREATE TABLE IF NOT EXISTS grupos (
                grupo_id INTEGER PRIMARY KEY AUTOINCREMENT,
                nombre TEXT NOT NULL,
                ciclo_escolar TEXT NOT NULL
            );
        """) # Tabla de grupos, el if not exists evita errores si la
tabla ya existe
        self.ejecutar_query("""
            CREATE TABLE IF NOT EXISTS alumnos (
                matricula TEXT PRIMARY KEY,
```

```

        nombre_completo TEXT NOT NULL,
        datos_contacto TEXT,
        email TEXT,
        grupo_id INTEGER,
        FOREIGN KEY (grupo_id) REFERENCES grupos(grupo_id)
    );
    """ # Tabla de alumnos, con clave primaria matricula y clave
foránea grupo_id
    #La diferencia entre text y varchar es que text no tiene
límite de longitud en SQLite
    self.ejecutar_query("""
        CREATE TABLE IF NOT EXISTS categorias_evaluacion (
            grupo_id INTEGER NOT NULL,
            nombre_categoria TEXT NOT NULL,
            peso_porcentual REAL NOT NULL,
            max_items INTEGER NOT NULL DEFAULT 1,
            PRIMARY KEY (grupo_id, nombre_categoria),
            FOREIGN KEY (grupo_id) REFERENCES grupos(grupo_id) ON
DELETE CASCADE
        );
    """)
    #El self.ejecutar_query se usa para ejecutar comandos SQL que
crean tablas
    self.ejecutar_query("""
        CREATE TABLE IF NOT EXISTS asistencia (
            matricula TEXT NOT NULL,
            fecha TEXT NOT NULL,
            estado TEXT NOT NULL,
            PRIMARY KEY (matricula, fecha),
            FOREIGN KEY (matricula) REFERENCES alumnos(matricula)
ON DELETE CASCADE
        );
    """)
    self.ejecutar_query("""
        CREATE TABLE IF NOT EXISTS calificaciones (
            matricula TEXT NOT NULL,
            categoria TEXT NOT NULL,
            fecha TEXT NOT NULL,
            valor REAL NOT NULL,
            PRIMARY KEY (matricula, categoria, fecha),
            FOREIGN KEY (matricula) REFERENCES alumnos(matricula)
ON DELETE CASCADE
    """)

```

```

        );
        """ #El text not null es para asegurar que esos campos
siempre tengan un valor

        self.ejecutar_query("""
            CREATE TABLE IF NOT EXISTS profesores (
profesor_id INTEGER PRIMARY KEY AUTOINCREMENT,
nombre_completo TEXT NOT NULL,
usuario TEXT UNIQUE NOT NULL, -- El usuario debe ser único
password TEXT NOT NULL,
email TEXT
            );
        """)

        #Métodos para conectar y desconectar de la base de datos
        #Crea una nueva conexión cada vez que se necesita ejecutar
una consulta.
        def _conectar(self):
            self._con = sqlite3.connect(self._db_file)
            self._con.execute("PRAGMA foreign_keys = ON;") #Clave foránea
activada
            self.cursor = self._con.cursor()
            return self._con

        #Se cierra la conexión después de que se realizó la operación
        def _desconectar(self, conn=None):
            connection = conn or self._con
            if connection:
                connection.close()
                if conn is None:
                    self._con = None

        #Método para ejecutar consultas SQL
        #Polimorfismo: Este método puede ser sobrescrito en subclases si se
necesita un comportamiento diferente.
        def ejecutar_query(self, query, params=()):
            try:
                conn = self._conectar()
                self.cursor.execute(query, params)
                conn.commit()
                if query.strip().upper().startswith(("SELECT",
"PRAGMA")):

```

```

        return self.cursor.fetchall()
    return True
except sqlite3.Error as e:
    print(f"Error al ejecutar consulta: {e}")
    return False
finally:
    self._desconectar(conn)

#Método para ejecutar múltiples consultas SQL
def ejecutar_queries_multiples(self, query: str, params_list:
list[tuple]):
    try:
        conn = self._conectar()
        self.cursor.executemany(query, params_list) #Ejecuta
múltiples queries con diferentes parámetros
        conn.commit()
        return True
    except sqlite3.Error as e:
        print(f"Error al ejecutar múltiples queries: {e}")
        conn.rollback() #Revierte los cambios en caso de error
        return False
    finally:
        self._desconectar(conn) #Desconecta la base de datos

# 1. GRUPO DAO
#Esta clase maneja las operaciones CRUD para la entidad Grupo.
#Hereda de BaseDAO para reutilizar la conexión y métodos comunes.
#CRUD: crear, leer, actualizar, eliminar
class GrupoDAO(BaseDAO):

    def crear_grupo(self, grupo: Grupo):
        query = "INSERT INTO grupos (nombre, ciclo_escolar) VALUES
(?, ?)" #Inserta un nuevo grupo
        params = (grupo.get_nombre(), grupo.get_ciclo())
        return self.ejecutar_query(query, params)

    def obtener_grupos(self): #Obtiene todos los grupos
        query = "SELECT grupo_id, nombre, ciclo_escolar FROM grupos
ORDER BY ciclo_escolar, nombre"
        return self.ejecutar_query(query)

```

```

    def buscar_grupo_por_nombre_ciclo(self, nombre, ciclo_escolar):
#Busca un grupo por nombre y ciclo escolar
        query = "SELECT grupo_id FROM grupos WHERE nombre = ? AND
ciclo_escolar = ?"
        resultado = self.ejecutar_query(query, (nombre,
ciclo_escolar))
        return resultado[0][0] if resultado else None

    def actualizar_grupo(self, grupo: Grupo): #Actualiza los datos de
un grupo existente
        query = "UPDATE grupos SET nombre = ?, ciclo_escolar = ?
WHERE grupo_id = ?"
        params = (grupo.get_nombre(), grupo.get_ciclo(),
grupo.get_id())
        return self.ejecutar_query(query, params)

    def eliminar_grupo(self, grupo_id): #Elimina un grupo por su ID
        query = "DELETE FROM grupos WHERE grupo_id = ?"
        return self.ejecutar_query(query, (grupo_id,))

# 2. ALUMNO DAO (CASO DE USO 2)
#Esta clase maneja las operaciones CRUD para la entidad Alumno.
#Hereda de BaseDAO para reutilizar la conexión y métodos comunes.
class AlumnoDAO(BaseDAO):

    def crear_alumno(self, alumno: Alumno, grupo_id): #Crea un nuevo
alumno asociado a un grupo
        query = "INSERT INTO alumnos (matricula, nombre_completo,
datos_contacto, email, grupo_id) VALUES (?, ?, ?, ?, ?)"
        params = (alumno.get_matricula(),
alumno.get_nombre_completo(), alumno.get_datos_contacto(),
alumno.get_email(), grupo_id)
        return self.ejecutar_query(query, params)

    def obtener_alumnos_por_grupo(self, grupo_id): #Obtiene todos los
alumnos de un grupo específico
        query = "SELECT matricula, nombre_completo, datos_contacto,
email FROM alumnos WHERE grupo_id = ? ORDER BY nombre_completo"
        return self.ejecutar_query(query, (grupo_id,))

    def actualizar_alumno(self, alumno: Alumno): #Actualiza los datos

```

```

de un alumno existente
        query = "UPDATE alumnos SET nombre_completo = ?,
datos_contacto = ?, email = ? WHERE matricula = ?"
        params = (alumno.get_nombre_completo(),
alumno.get_datos_contacto(), alumno.get_email(),
alumno.get_matricula())
        return self.ejecutar_query(query, params)

    def eliminar_alumno(self, matricula): #Elimina un alumno por su
matrícula
        query = "DELETE FROM alumnos WHERE matricula = ?"
        return self.ejecutar_query(query, (matricula,))

# 3. ASISTENCIA DAO (CASO DE USO 4)
#Esta clase maneja las operaciones CRUD para la entidad Asistencia.
class AsistenciaDAO(BaseDAO):

    def registrar_asistencia(self, asistencia_obj: Asistencia):
#Registra o actualiza la asistencia de un alumno en una fecha
específica
        fecha = asistencia_obj.get_fecha() or
date.today().isoformat()
        estado = asistencia_obj.get_estado() or "Presente"
        query = "REPLACE INTO asistencia (matricula, fecha, estado)
VALUES (?, ?, ?)"
        params = (asistencia_obj.get_matricula(), fecha, estado)
#Parámetros para la consulta
        return self.ejecutar_query(query, params)

    def obtener_asistencia_del_dia(self, fecha, grupo_id): #Obtiene
la asistencia de todos los alumnos de un grupo en una fecha
específica
        #Query que une alumnos con su asistencia del día y muestra
'Ausente' si no hay registro
        query = """
        SELECT
            A.matricula,
            A.nombre_completo,
            COALESCE(S.estado, 'Ausente')
        FROM alumnos A
        LEFT JOIN asistencia S
        ON A.matricula = S.matricula AND S.fecha = ?

```

```

        WHERE A.grupo_id = ?
        ORDER BY A.nombre_completo;
    """
    return self.ejecutar_query(query, (fecha, grupo_id))

    def obtener_porcentaje_asistencia_por_alumno(self, matricula:
str) -> float: #Calcula el porcentaje de asistencia (Asistencia +
Justificado) / Total de días registrados
    #La flecha indica que el método retorna un valor float
    """
    MÉTODO AGREGADO para CU6.
    Calcula el porcentaje de asistencia (Asistencia +
Justificado) / Total de días registrados.
    """
    query = """
    SELECT
        CAST(SUM(CASE WHEN estado IN ('Asistencia',
'Justificado') THEN 1 ELSE 0 END) AS REAL) AS asistencias_contables,
        COUNT(fecha) AS total_dias_registrados
    FROM asistencia
    WHERE matricula = ?;
    """
    resultado = self.ejecutar_query(query, (matricula,))
    #Si hay resultados, calcula el porcentaje
    if resultado and resultado[0]:
        asistencias, total_dias = resultado[0]
        if total_dias > 0:
            porcentaje = (asistencias / total_dias) * 100
            return round(porcentaje, 2) #Redondea a 2 decimales
    #Si no hay registros, retorna 100%
    return 100.0

# 4. CATEGORIAEVALUACION DAO (Ponderación CU3)
#Esta clase maneja las operaciones CRUD para la entidad
CategoriaEvaluacion.
class CategoriaEvaluacionDAO(BaseDAO):
    #Método para crear ponderaciones iniciales si no existen
    def crear_ponderacion_inicial(self, grupo_id: int) -> bool:
        try:
            conn = self._conectar()
            query_check = "SELECT COUNT(*) FROM categorias_evaluacion
WHERE grupo_id = ?"

```

```

        self.cursor.execute(query_check, (grupo_id,))
        count = self.cursor.fetchone()[0] #Cuenta cuántas
categorías existen para ese grupo
        #Si no hay categorías, inserta las predeterminadas

#Predeterminadas
        if count == 0:
            default_categories = [
                (grupo_id, 'Examen Final', 50.0, 1),
                (grupo_id, 'Tareas', 30.0, 1),
                (grupo_id, 'Participación', 20.0, 1),
            ]
            query_insert = """
INSERT INTO categorias_evaluacion (grupo_id,
nombre_categoria, peso_porcentual, max_items)
VALUES (?, ?, ?, ?)
"""
            self.cursor.executemany(query_insert,
default_categories) #Inserta múltiples categorías a la vez
            conn.commit() #Confirma los cambios
            return True
        except sqlite3.Error as e: #Maneja errores de la base de
datos
            print(f"Error al crear ponderaciones iniciales: {e}")
            conn.rollback() #Revierte los cambios en caso de error,
lo que significa que no se guardan los cambios realizados antes del
error

            return False
    finally:
        self._desconectar(conn)

    #Método para obtener categorías de evaluación por grupo
    def obtener_categorias_por_grupo(self, grupo_id):
        query = """
SELECT grupo_id, nombre_categoria, peso_porcentual, max_items
FROM categorias_evaluacion
WHERE grupo_id = ?
ORDER BY nombre_categoria
"""
        resultados = self.ejecutar_query(query, (grupo_id,))
        categorias = [
            CategoriaEvaluacion(r[0], r[1], r[2], r[3]) for r in

```



```

resultados #La r[] es cada fila del resultado
    ]
    return categorias

#Método para guardar ponderaciones (categorías) para un grupo
    def guardar_ponderaciones(self, categorias:
list[CategoriaEvaluacion], grupo_id: int):
    try:
        conn = self._conectar()

        #1. Eliminar las categorías existentes para ese grupo, se
eliminan primero para evitar duplicados
        self.cursor.execute("DELETE FROM categorias_evaluacion
WHERE grupo_id = ?", (grupo_id,))

        #2. Insertar las nuevas categorías
        if categorias:
            query = """
                INSERT INTO categorias_evaluacion (grupo_id,
nombre_categoria, peso_porcentual, max_items)
                VALUES (?, ?, ?, ?)
            """

            #Convertir lista de objetos a lista de tuplas
            params_list = []
            for c in categorias:
                #Asumimos que CategoriaEvaluacion tiene los
métodos get_x() correctos.
                params_list.append((
                    grupo_id,
                    c.get_nombre_categoria(),
                    c.get_peso_porcentual(),
                    c.get_max_items()
                ))

            self.cursor.executemany(query, params_list) #Inserta
múltiples categorías a la vez

            conn.commit() #Confirma los cambios
            return True
    except sqlite3.Error as e:
        print(f"Error al guardar ponderaciones: {e}")
        conn.rollback()

```

```

        return False
    finally:
        self._desconectar(conn)

# 5. CALIFICACION DAO (CASO DE USO 5)
#Clase que maneja las operaciones CRUD para la entidad Calificacion.
class CalificacionDAO(BaseDAO):
    #Método para registrar o actualizar una calificación
    def registrar_calificacion(self, calificacion: Calificacion):
        #Calificacion tiene un método to_tuple() que retorna
        (matricula, categoria, fecha, valor)
        query = """
        REPLACE INTO calificaciones (matricula, categoria, fecha,
valor)
        VALUES (?, ?, ?, ?)
        """
        params = (calificacion.get_matricula(),
calificacion.get_categoria(), calificacion.get_fecha() or
date.today().isoformat(), calificacion.get_valor())
        return self.ejecutar_query(query, params)
    #Método para obtener calificaciones por categoría en un grupo
    def obtener_calificaciones_por_categoria(self, grupo_id,
categoria):
        query = """
        SELECT
            A.matricula,
            A.nombre_completo,
            T1.valor
        FROM alumnos A
        LEFT JOIN calificaciones T1
        ON A.matricula = T1.matricula AND T1.categoria = ?
        WHERE A.grupo_id = ?
        ORDER BY A.nombre_completo;
        """
        #Esta consulta puede retornar múltiples filas si hay muchas
        notas por alumno.
        return self.ejecutar_query(query, (categoria, grupo_id))
    #Método para obtener todas las calificaciones de un alumno
    por categoría para calcular promedios
    def obtener_calificaciones_por_alumno_y_categoria(self,
matricula: str):
        query = """

```

```

        SELECT categoria, valor, fecha
        FROM calificaciones
        WHERE matricula = ?
        ORDER BY categoria, fecha DESC;
        """

        return self.ejecutar_query(query, (matricula,))

# 6. PROFESOR DAO (Autenticación)
#Clase que maneja las operaciones CRUD para la entidad Profesor.
class ProfesorDAO(BaseDAO):
    #Método para crear un nuevo profesor
    def crear_profesor(self, nombre, usuario, password, email=""):
        query = "INSERT INTO profesores (nombre_completo, usuario, password, email) VALUES (?, ?, ?, ?)"
        params = (nombre, usuario, password, email)
        #ejecutar_query siempre retorna True o False para operaciones de inserción/actualización
        return self.ejecutar_query(query, params)

    #Método para buscar un profesor por su nombre de usuario
    def buscar_profesor_por_usuario(self, usuario):
        #Busca un profesor por su nombre de usuario.
        query = "SELECT profesor_id, nombre_completo, password, email FROM profesores WHERE usuario = ?"
        resultado = self.ejecutar_query(query, (usuario,))
        #ejecutar_query retorna una lista de tuplas (fetchall),
        return resultado[0] if resultado else None

```

Logica

gestor_alumnos.py

```

#Archivo para la lógica de negocio relacionada con la gestión de alumnos, grupos, asistencia y calificaciones.
from Datos.dao import AlumnoDAO, AsistenciaDAO, GrupoDAO, CategoriaEvaluacionDAO, CalificacionDAO
from model import Alumno, Asistencia, Grupo, CategoriaEvaluacion, Calificacion
from datetime import date #Para manejo de fechas

# 1. GESTOR GRUPOS

```

```

#Clase para gestionar la lógica de negocio relacionada con los
grupos. Qué reglas de negocio aplicar en cada caso de uso.
class GestorGrupos:
#Métodos para gestionar grupos, aplicando las reglas de negocio
correspondientes.
    def __init__(self):
        self._grupo_dao = GrupoDAO()
        self._alumno_dao = AlumnoDAO() #Necesario para BR.2
#Método que retornan listas de grupos
    def obtener_lista_grupos(self):
        return self._grupo_dao.obtener_grupos()

#Método para agregar un nuevo grupo, validando reglas de negocio
    def agregar_nuevo_grupo(self, nombre, ciclo_escolar):
        nombre = nombre.strip() #Eliminar espacios en blanco al
inicio y final
        ciclo_escolar = ciclo_escolar.strip() #Eliminar espacios en
blanco al inicio y final
# Validaciones básicas
        if not nombre or not ciclo_escolar:
            return "Error: Nombre y Ciclo Escolar son obligatorios."

        #BR.1: El nombre del grupo debe ser único para ese Ciclo
Escolar.
        if self._grupo_dao.buscar_grupo_por_nombre_ciclo(nombre,
ciclo_escolar):
            return "Error: Ya existe un grupo con ese nombre en este
Ciclo Escolar."
#Crear el nuevo grupo
        nuevo_grupo = Grupo(None, nombre, ciclo_escolar)
        if self._grupo_dao.crear_grupo(nuevo_grupo):
            return "Éxito: Grupo registrado correctamente."
        else:
            return "Error: No se pudo guardar el grupo en la base de
datos."
#Método para actualizar datos de un grupo.
    def actualizar_datos_grupo(self, grupo_id, nombre,
ciclo_escolar):
        nombre = nombre.strip()
        ciclo_escolar = ciclo_escolar.strip()

        #BR.1 (Revisar si otro grupo ya tiene esa combinación

```

```

nombre/ciclo)
        id_existente =
self._grupo_dao.buscar_grupo_por_nombre_ciclo(nombre, ciclo_escolar)
        if id_existente and id_existente != grupo_id:
            return "Error: Otro grupo ya usa ese nombre y ciclo
escolar."
#Actualizar el grupo
        grupo_a_actualizar = Grupo(grupo_id, nombre, ciclo_escolar)
        if self._grupo_dao.actualizar_grupo(grupo_a_actualizar):
            return "Éxito: Grupo actualizado correctamente."
        else:
            return "Error: No se pudo actualizar el grupo."
#Método para eliminar un grupo.
        def eliminar_grupo(self, grupo_id):
            #BR.2: Un grupo no puede ser eliminado si tiene alumnos
registrados
            if self._alumno_dao.obtener_alumnos_por_grupo(grupo_id):
                return "Error: No se puede eliminar el grupo porque tiene
alumnos registrados."

            if self._grupo_dao.eliminar_grupo(grupo_id):
                return "Éxito: Grupo eliminado."
            else:
                return "Error: No se pudo eliminar el grupo."

#2. GESTOR ALUMNOS
#Clase para gestionar la lógica de negocio relacionada con los
alumnos.
#Polimorfismo: Esta clase puede ser extendida para diferentes tipos
de alumnos si es necesario.
class GestorAlumnos:
#Método para inicializar el gestor con el grupo actual
        def __init__(self, grupo_actual_id):
            self._alumno_dao = AlumnoDAO()
            self._grupo_actual_id = grupo_actual_id
#Método privado para verificar si una matrícula ya existe en el grupo
#Es privado porque es una función auxiliar interna.
        def _existe_matricula_en_grupo(self, matricula):
            alumnos_grupo =
self._alumno_dao.obtener_alumnos_por_grupo(self._grupo_actual_id)
            return any(a[0] == matricula for a in alumnos_grupo)
#Método para agregar un nuevo alumno, aplicando las reglas de negocio

```

```

    def agregar_nuevo_alumno(self, matricula, nombre, contacto,
email):
    #Si no se proporcionan matrícula o nombre, retorna error
    if not matricula or not nombre:
        return "Error: Matrícula y Nombre son obligatorios."

    if self._existe_matricula_en_grupo(matricula):
        return f"Error: La matrícula {matricula} ya existe en
este grupo."
#Crear el nuevo alumno
    nuevo_alumno = Alumno(matricula, nombre, contacto, email)

    if self._alumno_dao.crear_alumno(nuevo_alumno,
self._grupo_actual_id):
        return "Éxito: Alumno registrado correctamente."
    else:
        return "Error: No se pudo guardar en la base de datos."
#Método para obtener la lista de alumnos del grupo actual
    def obtener_lista_alumnos(self):
        return
self._alumno_dao.obtener_alumnos_por_grupo(self._grupo_actual_id)
#Método para actualizar datos de un alumno
    def actualizar_datos_alumno(self, matricula, nombre, contacto,
email):

    if not matricula or not nombre:
        return "Error: Nombre es obligatorio para la
actualización."

    alumno_a_actualizar = Alumno(matricula, nombre, contacto,
email)

    if self._alumno_dao.actualizar_alumno(alumno_a_actualizar):
        return "Éxito: Datos del alumno actualizados
correctamente."
    else:
        return "Error: No se pudo actualizar el alumno."
#Método para eliminar un alumno
    def eliminar_alumno(self, matricula):
    if self._alumno_dao.eliminar_alumno(matricula):
        return "Éxito: Alumno eliminado."
    else:

```

```

        return "Error: No se pudo eliminar el alumno."

# 3. GESTOR ASISTENCIA
#Clase para gestionar la lógica de negocio relacionada con la
asistencia de los alumnos.
#Polimorfismo: Esta clase puede ser extendida para diferentes tipos
de asistencia si es necesario.
class GestorAsistencia:
#Método para inicializar el gestor con el grupo actual
    def __init__(self, grupo_actual_id):
        self._asistencia_dao = AsistenciaDAO()
        self._alumno_dao = AlumnoDAO()
        self._grupo_actual_id = grupo_actual_id
#Método para registrar asistencia masiva (poner asistencia a todos)
    def registrar_asistencia_masiva(self,
fecha=date.today().strftime("%Y-%m-%d")):
        alumnos_data =
self._alumno_dao.obtener_alumnos_por_grupo(self._grupo_actual_id)
        matriculas = [a[0] for a in alumnos_data]
        registros_exitosos = 0
#Registrar asistencia para cada alumno
        for matricula in matriculas:
#Crear el objeto Asistencia con estado "Asistencia"
            asistencia = Asistencia(matricula, fecha, "Asistencia")
            if self._asistencia_dao.registrar_asistencia(asistencia):
                registros_exitosos += 1
#Retornar mensaje según resultados
            if len(matriculas) == 0:
                return "Advertencia: No hay alumnos en este grupo."
            elif registros_exitosos > 0:
                return "Éxito: Asistencia masiva registrada."
            else:
                return "Error: No se pudo registrar la asistencia."
#Método para actualizar el estado de asistencia de un alumno
    def actualizar_estado_asistencia(self, matricula, fecha,
nuevo_estado):
        if nuevo_estado not in ["Asistencia", "Ausente", "Retardo",
"Justificado"]:
            return "Error: Estado de asistencia inválido."
        #Crear el objeto Asistencia con el nuevo estado
        asistencia = Asistencia(matricula, fecha, nuevo_estado)
        if self._asistencia_dao.registrar_asistencia(asistencia):

```

```

        return "Éxito: Estado de asistencia actualizado."
    else:
        return "Error: No se pudo actualizar el estado en la base
de datos."
#Método para obtener la lista de asistencia del día para la UI
    def obtener_asistencia_para_ui(self,
fecha=date.today().strftime("%Y-%m-%d")):
        return self._asistencia_dao.obtener_asistencia_del_dia(fecha,
self._grupo_actual_id)

# 4. GESTOR CALIFICACIONES (CASO DE USO 3, 5 y 6)
#Clase para gestionar la lógica de negocio relacionada con las
calificaciones y evaluaciones.
class GestorCalificaciones:
    #Método para inicializar el gestor con el grupo actual
    def __init__(self, grupo_id):
        self._grupo_actual_id = grupo_id
        self._categoria_dao = CategoriaEvaluacionDAO()
        self._calificacion_dao = CalificacionDAO()
        #CU6: Promedio Final y Riesgo
        self._alumno_dao = AlumnoDAO()
        self._asistencia_dao = AsistenciaDAO()
        #Aseguramos la ponderación inicial
        self._categoria_dao.crear_ponderacion_inicial(grupo_id)

    #Métodos para gestionar categorías de evaluación
    def obtener_categorias_evaluacion(self):
        return
self._categoria_dao.obtener_categorias_por_grupo(self._grupo_actual_i
d)

    #Método para guardar las categorías de evaluación con sus
ponderaciones
    def guardar_categorias_evaluacion(self, categorias_data:
list[tuple]):
        #Validar la suma de pesos 100%
        try:
            total_peso = sum([float(peso) for _, peso, _ in
categorias_data])
        except (TypeError, ValueError):
            return "Error: Los pesos porcentuales deben ser
numéricos."

```



```

        if round(total_peso) != 100:
            return f"Error: La suma de las ponderaciones debe ser
100%, la suma actual es {total_peso:.1f}%."

        #Crear objetos del modelo CategoriaEvaluacion
        lista_modelos = []
        for nombre, peso, max_items in categorias_data:
            lista_modelos.append(
                CategoriaEvaluacion(self._grupo_actual_id, nombre,
float(peso), int(max_items))
            )

        #Guardar en la base de datos
        try:
            #Llamada al método guardando el orden (lista_modelos,
grupo_id)
            if
self._categoria_dao.guardar_ponderaciones(lista_modelos,
self._grupo_actual_id):
                #6. Guarda la estructura y recalcula todos los
promedios
                self._recalcular_promedios()
                return "Éxito: Estructura de evaluación guardada y
promedios recalculados exitosamente."
            else:
                return "Error: El DAO retornó un fallo al guardar la
estructura de evaluación."
        except Exception as e:
            return f"Error crítico al intentar guardar ponderaciones:
{e}"

#Método para recalcular promedios de todos los alumnos
def _recalcular_promedios(self):
    alumnos_data =
self._alumno_dao.obtener_alumnos_por_grupo(self._grupo_actual_id)
    categorias = self.obtener_categorias_evaluacion()
#Recorre cada alumno para recalcular su promedio
    for matricula, _, _, _ in alumnos_data:
        #Recalcula el promedio de cada alumno.
        self._calcular_promedio_alumno_ponderado(matricula,
categorias)

```

```

        print(f"Recalculo de promedios finalizado para grupo
{self._grupo_actual_id}.")
        return True
#Método para calcular el promedio ponderado de un alumno
    def _calcular_promedio_alumno_ponderado(self, matricula,
ponderaciones):
        #Obtener todas las calificaciones del alumno (categoría,
valor, fecha)
        calificaciones_alumno =
self._calificacion_dao.obtener_calificaciones_por_alumno_y_categoria(
matricula)

        #Reestructurar calificaciones por categoría para un acceso
más fácil
        notas_por_categoria = {}
        for categoria, valor, _ in calificaciones_alumno:
            if categoria not in notas_por_categoria:
                notas_por_categoria[categoria] = []
            notas_por_categoria[categoria].append(valor)
#Calcular el promedio ponderado
        suma_ponderada = 0.0
        peso_total_valido = 0.0
        #Recorrer cada categoría y aplicar las reglas de negocio
        for categoria in ponderaciones:
            nombre_cat = categoria.get_nombre_categoria()
            peso_cat = categoria.get_peso_porcentual()
            max_items = categoria.get_max_items()

            notas_categoria = notas_por_categoria.get(nombre_cat)

            if notas_categoria:
                #Ordenar notas de mayor a menor y seleccionar las
mejores según max_items
                notas_categoria.sort(reverse=True)
                notas_seleccionadas = notas_categoria[:max_items]

                #Calcular promedio de la categoría
                promedio_categoria = sum(notas_seleccionadas) /
len(notas_seleccionadas)

                suma_ponderada += promedio_categoria * (peso_cat /
100.0)

```

```

        peso_total_valido += peso_cat #Suma el peso solo si
hay al menos una nota registrada en la categoría

    if peso_total_valido == 0:
        return 0.0

    #Retornar el promedio ponderado final
    return suma_ponderada / (peso_total_valido / 100.0)

#Método para calcular promedios finales y estado de riesgo de todos
los alumnos
    def calcular_promedios_y_estado_final(self):
        alumnos =
self._alumno_dao.obtener_alumnos_por_grupo(self._grupo_actual_id)
        ponderaciones =
self._categoria_dao.obtener_categorias_por_grupo(self._grupo_actual_i
d)

        resultados_finales = []

        for matricula, nombre, _, _ in alumnos:
            #1. Calcular promedio ponderado (BR.14)
            promedio =
self._calcular_promedio_alumno_ponderado(matricula, ponderaciones)

            #2. Obtener porcentaje de asistencia
            porcentaje_asistencia =
self._asistencia_dao.obtener_porcentaje_asistencia_por_alumno(matricu
la)

            #3. Determinar estado de riesgo (BR.12) y redondear
(BR.17)

            #Redondeo a 2 decimales
            promedio_redondeado = round(promedio, 2)
            porcentaje_asistencia_redondeado =
round(porcentaje_asistencia, 2)

            estado_riesgo = "Normal"

            #BR.12: Riesgo Académico < 7.0
            if promedio_redondeado < 7.0:
                estado_riesgo = "Riesgo Académico"

```

```

        #BR.12: Riesgo Asistencia < 80.0
        if porcentaje_asistencia_redondeado < 80.0:
            if estado_riesgo == "Riesgo Académico":
                estado_riesgo = "Riesgo Académico y Asistencia"
            else:
                estado_riesgo = "Riesgo Asistencia"
        #Agregar resultado a la lista final
        resultados_finales.append((
            matricula,
            nombre,
            promedio_redondeado,
            porcentaje_asistencia_redondeado,
            estado_riesgo
        ))

    return resultados_finales

#Método para obtener alumnos con calificaciones en una categoría
específica
    def obtener_alumnos_con_calificaciones(self, categoria):
        #Retorna: [(matricula, nombre, valor), ...]
        return
self._calificacion_dao.obtener_calificaciones_por_categoria(self._grupo_
po_actual_id, categoria)
#Método para registrar una calificación para un alumno
    def registrar_calificacion(self, matricula, categoria, valor):
        #Validación de rangos
        try:
            valor_num = float(valor)
        except (TypeError, ValueError):
            return "Error: La calificación debe ser un valor
numérico."

        if not (0.0 <= valor_num <= 10.0):
            return "Error: La calificación debe estar entre 0 y 10."

        #Construir el modelo de calificación
        nueva_calificacion = Calificacion(matricula, categoria,
valor_num)
        try:
            saved =

```

```

self._calificacion_dao.registrar_calificacion(nueva_calificacion)
    except Exception as e:
        print(f"Error al registrar calificación: {e}")
        saved = False

    if saved:
        #Recalcular promedios para actualizar estado de riesgo
        try:
            self._recalcular_promedios()
        except Exception as e:
            #Alerta pero no bloquea el guardado
            print(f"Advertencia: Fallo en el recálculo de
promedios post-guardado: {e}")
            pass
        return "Éxito: Calificación registrada."
    else:
        return "Error: No se pudo registrar la calificación."

```

gestor_autenticacion.py

```

# Logica/gestor_autenticacion.py
from Datos.dao import ProfesorDAO

class GestorAutenticacion:
    def __init__(self):
        self._profesor_dao = ProfesorDAO()

    def autenticar_profesor(self, usuario, password):
        profesor = self._profesor_dao.buscar_profesor_por_usuario(usuario)

        if not profesor:
            return {
                "autenticado": False,
                "mensaje": "Usuario no encontrado"
            }

        # profesor[2] es la contraseña almacenada
        if profesor[2] != password:
            return {
                "autenticado": False,
                "mensaje": "Contraseña incorrecta"
            }

        return {
            "autenticado": True,

```

```

        "mensaje": f"Bienvenido, {profesor[1]}",
        "profesor_id": profesor[0],
        "nombre": profesor[1]
    }

def registrar_profesor(self, nombre, usuario, password, email=""):
    """Registra un nuevo profesor en el sistema."""

    # 1. Verificar si el usuario ya existe (Regla de unicidad)
    if self._profesor_dao.buscar_profesor_por_usuario(usuario):
        return {
            "exito": False,
            "mensaje": "El nombre de usuario ya está en uso. Por favor, elija otro."
        }

    # 2. Registrar el profesor
    try:
        self._profesor_dao.crear_profesor(nombre, usuario, password, email)
        return {
            "exito": True,
            "mensaje": "¡Registro exitoso! Ya puedes iniciar sesión."
        }
    except Exception as e:
        # Manejo de errores de base de datos
        return {
            "exito": False,
            "mensaje": f"Error al guardar en la base de datos: {str(e)}"
        }

```

gestor_calificaciones.py

```

#Archivo para gestionar las calificaciones y ponderaciones
from Datos.dao import AsistenciaDAO, CategoriaEvaluacionDAO,
CalificacionDAO, AlumnoDAO
from model import Calificacion, CategoriaEvaluacion
from datetime import date

#Clase para gestionar las calificaciones y ponderaciones
class GestorCalificaciones:
    #Método constructor
    def __init__(self, grupo_id :int):
        self._grupo_actual_id = grupo_id
        self._calificacion_dao = CalificacionDAO()
        self._ponderacion_dao = CategoriaEvaluacionDAO()
        self._alumno_dao = AlumnoDAO() #Necesario para obtener
alumnos y sus matrículas

```

```

        self._asistencia_dao = AsistenciaDAO()
        self._ponderacion_dao.crear_ponderacion_inicial(grupo_id)
#Asegura ponderación inicial

#LÓGICA CU3: ADMINISTRAR PONDERACIONES DE EVALUACIÓN
#Método para obtener categorías
    def obtener_categorias(self):
        return
self._ponderacion_dao.obtener_categorias_por_grupo(self._grupo_actual_id)
#Método para guardar ponderaciones
    def guardar_ponderaciones(self, categorias:
list[CategoriaEvaluacion]):
        #BR.5: Debe haber al menos una categoría
        if not categorias:
            return "Error: Debe definir al menos una categoría de
evaluación."

        #BR.4: La suma de los pesos debe ser 100.0
        suma_pesos = sum(c.get_peso_porcentual() for c in categorias)
        if abs(suma_pesos - 100.0) > 0.01: #La tolerancia de 0.01
para evitar errores de punto flotante
            return f"Error: La suma de los pesos porcentuales debe
ser 100%. Suma actual: {suma_pesos:.2f}% ."
        for categoria in categorias:
            pass

        if
self._ponderacion_dao.guardar_ponderaciones(self._grupo_actual_id,
categorias):
            return "Éxito: Ponderaciones guardadas correctamente."
        else:
            return "Error: No se pudieron guardar las ponderaciones
en la base de datos."

#LÓGICA CU5: REGISTRAR CALIFICACIONES
#Método para obtener calificaciones por categoría
    def obtener_calificaciones_por_categoria(self, categoria_nombre):
        return
self._calificacion_dao.obtener_calificaciones_por_categoria(
        self._grupo_actual_id, categoria_nombre
    )

```

```

#Método para registrar calificación
def registrar_calificacion(self, matricula, categoria, valor,
fecha=None):
    try:
        valor = float(valor)
    except ValueError:
        return "Error: La calificación debe ser un valor
numérico."

    #BR.13: Todas las calificaciones deben ser numéricas y estar
dentro de la escala (0-10)
    if not (0.0 <= valor <= 10.0):
        return "Error: La calificación debe estar entre 0.0 y
10.0 "

    #Usar la fecha de hoy si no se proporciona
    fecha_registro = fecha if fecha is not None else
date.today().isoformat()

    nueva_calificacion = Calificacion(matricula, categoria,
valor, fecha_registro)

    if
self._calificacion_dao.registrar_calificacion(nueva_calificacion):
#Así se registra en la BD
        return "Éxito: Calificación registrada."
    else:
        return "Error: No se pudo registrar la calificación."

# LÓGICA BR.14/BR.15: CÁLCULO DE PROMEDIOS
#Método para calcular el promedio final de un alumno
def calcular_promedio_final(self, matricula):
    categorias = self.obtener_categorias()
    if not categorias: #Si no hay categorías definidas
        return 0.0

    #1. Obtener todas las calificaciones del alumno para cada
categoría
    promedio_ponderado = 0.0 #Valor inicial
    peso_total_valido = 0.0 #Para manejar casos donde no hay
calificaciones
    for categoria in categorias:
        nota_categoria = 0.0 #Valor por defecto si no hay

```



```

calificaciones
        promedio_ponderado += (nota_categoria *
categoria.get_peso_porcentual()) #Ponderar
        peso_total_valido += categoria.get_peso_porcentual()
#Acumular peso válido
        return promedio_ponderado / 100.0

#Función que junta todo para el reporte/vista final

def obtener_resumen_calificaciones_grupo(self):

    alumnos =
self._alumno_dao.obtener_alumnos_por_grupo(self._grupo_actual_id)
    categorias = self.obtener_categorias()
    #Datos a retornar
    datos_grupo = []
    for alumno in alumnos:
        matricula = alumno.get_matricula()
        nombre = alumno.get_nombre_completo()

        registro = {
            'matricula': matricula,
            'nombre_completo': nombre,
            #Promedio final
            'promedio_final':
self.calcular_promedio_final(matricula)
        }

        # Agregar la nota individual por cada categoría
        for categoria in categorias:
            pass

        datos_grupo.append(registro)

    return datos_grupo, categorias

```

gestor_exportacion.py

```

#logica/gestor_expotacion.py
import pandas as pd

```

```

import sqlite3
from datetime import datetime
import os

class GestorExportaciones:
    """Gestiona la exportación de datos a Excel para todos los módulos."""

    def __init__(self, db_file='directaula.db'):
        self._db_file = db_file

    def exportar_grupo_completo(self, grupo_id, nombre_grupo, directorio_destino=None):
        """
        Exporta todos los datos de un grupo a un archivo Excel con múltiples hojas.
        Retorna: (éxito, mensaje, ruta_archivo)
        """
        try:
            # Crear nombre del archivo
            timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
            nombre_archivo = f"DirectAula_{nombre_grupo.replace(' ',
            '_')}_{timestamp}.xlsx"

            if directorio_destino:
                ruta_completa = os.path.join(directorio_destino, nombre_archivo)
            else:
                ruta_completa = nombre_archivo

            # Crear ExcelWriter
            with pd.ExcelWriter(ruta_completa, engine='openpyxl') as writer:
                # 1. Hoja: Información del Grupo
                self._exportar_info_grupo(writer, grupo_id, nombre_grupo)

                # 2. Hoja: Lista de Alumnos
                self._exportar_alumnos(writer, grupo_id)

                # 3. Hoja: Asistencia Resumen
                self._exportar_asistencia_resumen(writer, grupo_id)

                # 4. Hoja: Calificaciones por Categoría
                self._exportar_calificaciones_categorias(writer, grupo_id)

                # 5. Hoja: Resultados Finales
                self._exportar_resultados_finales(writer, grupo_id)

                # 6. Hoja: Asistencia Detallada
                self._exportar_asistencia_detallada(writer, grupo_id)

                # 7. Hoja: Calificaciones Detalladas
                self._exportar_calificaciones_detalladas(writer, grupo_id)

            return True, f"Archivo exportado exitosamente: {nombre_archivo}",
            ruta_completa

```

```

except Exception as e:
    return False, f"Error al exportar: {str(e)}", None

def _exportar_info_grupo(self, writer, grupo_id, nombre_grupo):
    """Exporta información básica del grupo."""
    try:
        conn = sqlite3.connect(self._db_file)

        # Obtener información del grupo
        query_grupo = "SELECT nombre, ciclo_escolar FROM grupos WHERE grupo_id = ?"
        grupo_data = pd.read_sql_query(query_grupo, conn, params=(grupo_id,))

        # Contar alumnos
        query_alumnos = "SELECT COUNT(*) as total FROM alumnos WHERE grupo_id = ?"
        total_alumnos = pd.read_sql_query(query_alumnos, conn,
params=(grupo_id,)).iloc[0]['total']

        # Crear DataFrame con información
        info_data = {
            'Campo': ['Nombre del Grupo', 'Ciclo Escolar', 'Fecha de Exportación',
'Total de Alumnos'],
            'Valor': [
                nombre_grupo,
                grupo_data.iloc[0]['ciclo_escolar'] if not grupo_data.empty else
'N/A',

                datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                total_alumnos
            ]
        }

        df_info = pd.DataFrame(info_data)
        df_info.to_excel(writer, sheet_name='Información del Grupo', index=False)

        # Ajustar ancho de columnas
        worksheet = writer.sheets['Información del Grupo']
        worksheet.column_dimensions['A'].width = 20
        worksheet.column_dimensions['B'].width = 30

    except Exception as e:
        print(f"Error en info grupo: {e}")
    finally:
        if 'conn' in locals():
            conn.close()

def _exportar_alumnos(self, writer, grupo_id):
    """Exporta la lista completa de alumnos."""
    try:
        conn = sqlite3.connect(self._db_file)
        query = """
SELECT matricula, nombre_completo, datos_contacto, email
FROM alumnos
WHERE grupo_id = ?

```

```

ORDER BY nombre_completo
"""

df_alumnos = pd.read_sql_query(query, conn, params=(grupo_id,))

if not df_alumnos.empty:
    df_alumnos.to_excel(writer, sheet_name='Lista de Alumnos', index=False)

    # Ajustar formatos
    worksheet = writer.sheets['Lista de Alumnos']
    worksheet.column_dimensions['A'].width = 15 # Matrícula
    worksheet.column_dimensions['B'].width = 30 # Nombre
    worksheet.column_dimensions['C'].width = 20 # Contacto
    worksheet.column_dimensions['D'].width = 25 # Email

except Exception as e:
    print(f"Error en alumnos: {e}")
finally:
    if 'conn' in locals():
        conn.close()

def _exportar_asistencia_resumen(self, writer, grupo_id):
    """Exporta resumen de asistencia por alumno."""
    try:
        conn = sqlite3.connect(self._db_file)
        query = """
SELECT
    a.matricula,
    a.nombre_completo,
    COUNT(CASE WHEN s.estado IN ('Presente', 'Asistencia', 'Justificado')
THEN 1 END) as dias_asistidos,
    COUNT(CASE WHEN s.estado = 'Ausente' THEN 1 END) as dias_ausente,
    COUNT(CASE WHEN s.estado = 'Retardo' THEN 1 END) as dias_retardo,
    COUNT(CASE WHEN s.estado = 'Justificado' THEN 1 END) as
dias_justificado,
    COUNT(s.estado) as total_registros,
    ROUND(
        (COUNT(CASE WHEN s.estado IN ('Presente', 'Asistencia',
'Justificado') THEN 1 END) * 100.0 /
        NULLIF(COUNT(s.estado), 0)), 2
    ) as porcentaje_asistencia
FROM alumnos a
LEFT JOIN asistencia s ON a.matricula = s.matricula
WHERE a.grupo_id = ?
GROUP BY a.matricula, a.nombre_completo
ORDER BY a.nombre_completo
"""
        df_asistencia = pd.read_sql_query(query, conn, params=(grupo_id,))

        if not df_asistencia.empty:
            df_asistencia.to_excel(writer, sheet_name='Asistencia Resumen',
index=False)

```

```

        worksheet = writer.sheets['Asistencia Resumen']
        columns = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']
        widths = [15, 30, 12, 12, 12, 12, 12, 15]

        for col, width in zip(columns, widths):
            worksheet.column_dimensions[col].width = width

    except Exception as e:
        print(f"Error en asistencia resumen: {e}")
    finally:
        if 'conn' in locals():
            conn.close()

def _exportar_calificaciones_categorias(self, writer, grupo_id):
    """Exporta calificaciones organizadas por categoría."""
    try:
        conn = sqlite3.connect(self._db_file)

        # Obtener categorías del grupo
        query_categorias = """
        SELECT DISTINCT nombre_categoria
        FROM categorias_evaluacion
        WHERE grupo_id = ?
        """

        categorias_df = pd.read_sql_query(query_categorias, conn,
params=(grupo_id,))

        if categorias_df.empty:
            # Si no hay categorías, crear una hoja vacía
            empty_df = pd.DataFrame({'Mensaje': ['No hay categorías de evaluación
definidas para este grupo']})
            empty_df.to_excel(writer, sheet_name='Calificaciones', index=False)
            return

        # Para cada categoría, obtener calificaciones
        for _, cat_row in categorias_df.iterrows():
            categoria = cat_row['nombre_categoria']
            query_calificaciones = """
            SELECT
                a.matricula,
                a.nombre_completo,
                c.valor as calificacion,
                c.fecha
            FROM alumnos a
            LEFT JOIN calificaciones c ON a.matricula = c.matricula AND c.categoria
            = ?
            WHERE a.grupo_id = ?
            ORDER BY a.nombre_completo, c.fecha DESC
            """
            df_calif = pd.read_sql_query(query_calificaciones, conn,
params=(categoria, grupo_id))

```

```

        if not df_calif.empty:
            # Tomar solo la calificación más reciente por alumno
            df_calif_reciente = df_calif.drop_duplicates(subset=['matricula'],
keep='first')

            df_calif_reciente = df_calif_reciente[['matricula',
'nombre_completo', 'calificacion']]
            df_calif_reciente.rename(columns={'calificacion': f'Calificación
{categoria}'}, inplace=True)

            # Nombre de hoja limitado a 31 caracteres
            sheet_name = categoria[:31]
            df_calif_reciente.to_excel(writer, sheet_name=sheet_name,
index=False)

            worksheet = writer.sheets[sheet_name]
            worksheet.column_dimensions['A'].width = 15
            worksheet.column_dimensions['B'].width = 30
            worksheet.column_dimensions['C'].width = 15

    except Exception as e:
        print(f"Error en calificaciones categorías: {e}")
    finally:
        if 'conn' in locals():
            conn.close()

def _exportar_resultados_finales(self, writer, grupo_id):
    """Exporta resultados finales con promedios y estados de riesgo."""
    try:
        conn = sqlite3.connect(self._db_file)

        # Obtener alumnos del grupo
        query_alumnos = """
SELECT matricula, nombre_completo
FROM alumnos
WHERE grupo_id = ?
ORDER BY nombre_completo
"""
        alumnos_df = pd.read_sql_query(query_alumnos, conn, params=(grupo_id,))

        if alumnos_df.empty:
            empty_df = pd.DataFrame({'Mensaje': ['No hay alumnos en este grupo']})
            empty_df.to_excel(writer, sheet_name='Resultados Finales', index=False)
            return

        # Obtener porcentaje de asistencia por alumno
        query_asistencia = """
SELECT
    a.matricula,
    ROUND(
        (COUNT(CASE WHEN s.estado IN ('Presente', 'Asistencia',
'Justificado') THEN 1 END) * 100.0 /
        NULLIF(COUNT(s.estado), 0)), 2

```

```

        ) as porcentaje_asistencia
    FROM alumnos a
    LEFT JOIN asistencia s ON a.matricula = s.matricula
    WHERE a.grupo_id = ?
    GROUP BY a.matricula
    """

    asistencia_df = pd.read_sql_query(query_asistencia, conn,
params=(grupo_id,))

    # Obtener promedio de calificaciones por alumno
    query_promedios = """
    SELECT
        a.matricula,
        ROUND(AVG(c.valor), 2) as promedio_final
    FROM alumnos a
    LEFT JOIN calificaciones c ON a.matricula = c.matricula
    WHERE a.grupo_id = ?
    GROUP BY a.matricula
    """

    promedios_df = pd.read_sql_query(query_promedios, conn, params=(grupo_id,))

    # Combinar todos los datos
    resultados_df = alumnos_df.merge(asistencia_df, on='matricula', how='left')
    resultados_df = resultados_df.merge(promedios_df, on='matricula',
how='left')

    # Calcular estado de riesgo
    def calcular_estado_riesgo(row):
        promedio = row['promedio_final'] if pd.notna(row['promedio_final']) else 0

        asistencia = row['porcentaje_asistencia'] if
pd.notna(row['porcentaje_asistencia']) else 0

        if promedio < 7.0 and asistencia < 80:
            return 'Riesgo Académico y Asistencia'
        elif promedio < 7.0:
            return 'Riesgo Académico'
        elif asistencia < 80:
            return 'Riesgo Asistencia'
        else:
            return 'Normal'

    resultados_df['estado_riesgo'] = resultados_df.apply(calcular_estado_riesgo,
axis=1)

    resultados_df['promedio_final'] = resultados_df['promedio_final'].fillna(0)
    resultados_df['porcentaje_asistencia'] =
resultados_df['porcentaje_asistencia'].fillna(0)

    # Renombrar columnas para mejor presentación
    resultados_df.columns = ['Matrícula', 'Nombre Completo', 'Asistencia (%)',
'Promedio Final', 'Estado de Riesgo']

```

```

        resultados_df.to_excel(writer, sheet_name='Resultados Finales', index=False)

        worksheet = writer.sheets['Resultados Finales']
        columns = ['A', 'B', 'C', 'D', 'E']
        widths = [15, 30, 15, 15, 25]

        for col, width in zip(columns, widths):
            worksheet.column_dimensions[col].width = width

    except Exception as e:
        print(f"Error en resultados finales: {e}")
    finally:
        if 'conn' in locals():
            conn.close()

def _exportar_asistencia_detallada(self, writer, grupo_id):
    """Exporta el registro detallado de asistencia."""
    try:
        conn = sqlite3.connect(self._db_file)
        query = """
        SELECT
            a.matricula,
            a.nombre_completo,
            s.fecha,
            s.estado
        FROM alumnos a
        JOIN asistencia s ON a.matricula = s.matricula
        WHERE a.grupo_id = ?
        ORDER BY s.fecha DESC, a.nombre_completo
        """
        df_detalle = pd.read_sql_query(query, conn, params=(grupo_id,))

        if not df_detalle.empty:
            df_detalle.to_excel(writer, sheet_name='Asistencia Detallada',
index=False)

            worksheet = writer.sheets['Asistencia Detallada']
            worksheet.column_dimensions['A'].width = 15
            worksheet.column_dimensions['B'].width = 30
            worksheet.column_dimensions['C'].width = 12
            worksheet.column_dimensions['D'].width = 12

    except Exception as e:
        print(f"Error en asistencia detallada: {e}")
    finally:
        if 'conn' in locals():
            conn.close()

def _exportar_calificaciones_detalladas(self, writer, grupo_id):
    """Exporta todas las calificaciones detalladas."""
    try:
        conn = sqlite3.connect(self._db_file)

```



```

        query = """
        SELECT
            a.matricula,
            a.nombre_completo,
            c.categoria,
            c.valor as calificacion,
            c.fecha
        FROM alumnos a
        JOIN calificaciones c ON a.matricula = c.matricula
        WHERE a.grupo_id = ?
        ORDER BY a.nombre_completo, c.categoria, c.fecha DESC
        """

        df_calif_detalle = pd.read_sql_query(query, conn, params=(grupo_id,))

        if not df_calif_detalle.empty:
            df_calif_detalle.to_excel(writer, sheet_name='Calificaciones
Detalladas', index=False)

            worksheet = writer.sheets['Calificaciones Detalladas']
            worksheet.column_dimensions['A'].width = 15
            worksheet.column_dimensions['B'].width = 30
            worksheet.column_dimensions['C'].width = 20
            worksheet.column_dimensions['D'].width = 12
            worksheet.column_dimensions['E'].width = 12

        except Exception as e:
            print(f"Error en calificaciones detalladas: {e}")
        finally:
            if 'conn' in locals():
                conn.close()

```

Presentacion

seleccion_grupo.py

```

# presentacion/seleccion_grupo.py
from PyQt5.QtWidgets import (
    QDialog, QComboBox, QVBoxLayout, QLabel,
    QDialogButtonBox, QMessageBox
)
from Logica.gestor_alumnos import GestorGrupos

class SeleccionGrupo(QDialog):
    """Obliga al usuario a seleccionar un Grupo antes de continuar."""

    def __init__(self, titulo_accion, parent=None):
        super().__init__(parent)
        self.setWindowTitle(f"Seleccionar Grupo - {titulo_accion}")

```

```

self.resize(300, 150)
self._gestor_grupos = GestorGrupos()
self._grupo_seleccionado_id = None
self._grupos_disponibles = {} # Diccionario: {nombre_completo: id}

self._inicializar_ui(titulo_accion)
self._cargar_grupos()

def _inicializar_ui(self, titulo_accion):
    layout = QVBoxLayout(self)

    lbl_instruccion = QLabel(f"Por favor, seleccione el grupo para {titulo_accion}
:")

    lbl_instruccion.setObjectName("subtitulo")
    layout.addWidget(lbl_instruccion)

    self.combo_grupos = QComboBox()
    layout.addWidget(self.combo_grupos)

    self.botones = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel,
self)

    self.botones.accepted.connect(self._aceptar_seleccion)
    self.botones.rejected.connect(self.reject)
    layout.addWidget(self.botones)

def _cargar_grupos(self):
    """Carga los grupos disponibles desde la BLL en el ComboBox."""
    grupos = self._gestor_grupos.obtener_lista_grupos() # Retorna: [id, nombre,
ciclo]

    if not grupos:
        QMessageBox.warning(self, "Advertencia",
            "No hay grupos registrados. Debe crear uno primero en el CU1."
        )
        self.botones.button(QDialogButtonBox.Ok).setEnabled(False)
        return

    self.combo_grupos.clear()

    for grupo_id, nombre, ciclo in grupos:
        nombre_display = f"{nombre} ({ciclo})"
        self.combo_grupos.addItem(nombre_display)
        self._grupos_disponibles[nombre_display] = grupo_id

def _aceptar_seleccion(self):
    """Valida la selección y guarda el ID."""
    nombre_seleccionado = self.combo_grupos.currentText()
    if nombre_seleccionado in self._grupos_disponibles:
        self._grupo_seleccionado_id = self._grupos_disponibles[nombre_seleccionado]
        self.accept()
    else:
        QMessageBox.critical(self, "Error", "Debe seleccionar un grupo válido.")

```

```

def get_grupo_id(self):
    """Retorna el ID del grupo seleccionado."""
    return self._grupo_seleccionado_id

```

ventana_alumnos.py

```

# presentacion/ventana_alumnos.py
import sys
from PyQt5.QtWidgets import (
    QApplication, QWidget, QVBoxLayout, QHBoxLayout, QLineEdit, QTableWidgetItem,
    QPushButton, QMessageBox, QDialog, QFormLayout, QDialogButtonBox,
    QLabel, QTableWidgetItem, QHeaderView, QStyleFactory, QFileDialog
)

from Logica.gestor_alumnos import GestorAlumnos
from Logica.gestor_exportacion import GestorExportaciones
# =====
# CLASE DE DIÁLOGO PARA AGREGAR/EDITAR ALUMNO
# =====
class DialogoAlumno(QDialog):
    """Formulario reutilizable para Agregar (FA.1) o Editar (FA.2) Alumno."""
    def __init__(self, datos_alumno=None, parent=None):
        super().__init__(parent)
        self.datos_alumno = datos_alumno
        self.setWindowTitle("Registrar Alumno" if not datos_alumno else "Editar Alumno")
        self._inicializar_ui(datos_alumno)

    def _inicializar_ui(self, datos_alumno):
        layout = QFormLayout()

        self.campo_matricula = QLineEdit()
        self.campo_nombre = QLineEdit()
        self.campo_contacto = QLineEdit()
        self.campo_email = QLineEdit()
        if datos_alumno:
            # datos_alumno = [matricula, nombre, contacto, email]
            self.campo_matricula.setText(datos_alumno[0])
            self.campo_matricula.setEnabled(False) # No se puede cambiar la matrícula
            self.campo_nombre.setText(datos_alumno[1])
            self.campo_contacto.setText(datos_alumno[2])
            self.campo_email.setText(datos_alumno[3]) #

        layout.addRow("Matrícula ", self.campo_matricula)
        layout.addRow("Nombre Completo ", self.campo_nombre)
        layout.addRow("Datos de Contacto", self.campo_contacto)
        layout.addRow("Email", self.campo_email)

        # botones
        self.botones = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel,
self)

```

```

        self.botonones.accepted.connect(self.accept)
        self.botonones.rejected.connect(self.reject)
        layout.addRow(self.botonones)
        self.setLayout(layout)

    def get_data(self):
        """Retorna los 4 campos obligatorios y el email."""
        return (
            self.campo_matricula.text().strip(),
            self.campo_nombre.text().strip(),
            self.campo_contacto.text().strip(),
            self.campo_email.text().strip()
        )

# =====
# VENTANA PRINCIPAL
# =====
class VentanaAlumnos(QWidget):
    def __init__(self, grupo_id, nombre_grupo):
        super().__init__()

        self._gestor_exportaciones = GestorExportaciones()
        self._grupo_id = grupo_id
        self._nombre_grupo = nombre_grupo # Guardamos el nombre del grupo
        self.setWindowTitle(f"DirectAula - Alumnos del grupo: {self._nombre_grupo}")
        self.resize(800, 400)
        self.gestor = GestorAlumnos(self._grupo_id)
        self._inicializar_ui(self._nombre_grupo)
        self._cargar_datos()

    def _inicializar_ui(self, nombre_grupo):
        main_layout = QVBoxLayout()
        lbl_titulo = QLabel(f"Gestión de Alumnos: {nombre_grupo}")
        lbl_titulo.setObjectName("titulo_principal")
        main_layout.addWidget(lbl_titulo)

        # --- SECCIÓN BÚSQUEDA Y ACCIONES ---
        self.lbl_subtitulo_acciones = QLabel("Búsqueda y acciones rápidas")
        self.lbl_subtitulo_acciones.setProperty("class", "subtitulo") # Aplica el estilo
#003366, negritas, 16px
        main_layout.addWidget(self.lbl_subtitulo_acciones)
        top_bar_layout = QHBoxLayout()
        # 1. Campo de Búsqueda (AC-2)

        self.campo_busqueda = QLineEdit()
        self.campo_busqueda.setPlaceholderText("Buscar por nombre o matrícula ")
        self.campo_busqueda.textChanged.connect(self._cargar_datos)
        top_bar_layout.addWidget(self.campo_busqueda, 1)

        # 2. Botones CRUD y Exportar
        self.btn_agregar = QPushButton("✚ Agregar")
        self.btn_agregar.setObjectName("btn_agregar")
        self.btn_agregar.clicked.connect(lambda: self._mostrar_formulario(None))

```

```

self.btn_editar = QPushButton("✎ Editar")
self.btn_editar.setObjectName("btn_editar")
self.btn_editar.clicked.connect(self._mostrar_formulario_editar)

self.btn_eliminar = QPushButton("🗑 Eliminar")
self.btn_eliminar.setObjectName("btn_eliminar")
self.btn_eliminar.clicked.connect(self._eliminar_alumno_seleccionado)

self.btn_exportar = QPushButton("📄 Exportar")
self.btn_exportar.setObjectName("btn_exportar")
self.btn_exportar.clicked.connect(self._exportar_excel)

top_bar_layout.addWidget(self.btn_agregar, 0)
top_bar_layout.addWidget(self.btn_editar, 0)
top_bar_layout.addWidget(self.btn_eliminar, 0)
top_bar_layout.addWidget(self.btn_exportar, 0)
main_layout.addLayout(top_bar_layout)

# --- SECCIÓN LISTA DE ALUMNOS ---
self.lbl_subtitulo_lista = QLabel("Lista de alumnos")
self.lbl_subtitulo_lista.setProperty("class", "subtitulo") # Aplica el estilo
#003366, negritas, 16px
main_layout.addWidget(self.lbl_subtitulo_lista)

# 3. Tabla de alumnos (R)
self.tabla_alumnos = QTableWidget()
self.tabla_alumnos.setColumnCount(4)
self.tabla_alumnos.setHorizontalHeaderLabels(["Matrícula", "Nombre Completo",
"Contacto", "Email"])
self.tabla_alumnos.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
main_layout.addWidget(self.tabla_alumnos)
self.setLayout(main_layout)

def _cargar_datos(self):
    """Muestra los datos obtenidos de la BLL en la tabla y aplica filtrado
(AC-2)."""
    datos = self.gestor.obtener_lista_alumnos()
    self.tabla_alumnos.setRowCount(0)
    busqueda_texto = self.campo_busqueda.text().lower()
    fila_indice = 0

    for alumno_data in datos:
        # alumno_data es: [matricula, nombre, contacto, email, grupo_id]

        # Filtrado rápido por matrícula o nombre (AC-2)
        if busqueda_texto in alumno_data[0].lower() or busqueda_texto in
alumno_data[1].lower():

            self.tabla_alumnos.insertRow(fila_indice)
            # Insertamos los 4 campos (Matrícula, Nombre, Contacto, Email)
            for columna, valor in enumerate(alumno_data[:4]):

```

```

        display_valor = str(valor) if valor is not None else ""
        item = QTableWidgetItem(display_valor)
        self.tabla_alumnos.setItem(fila_indice, columna, item)
        fila_indice += 1

    def _get_cell_text_safe(self, row, col):
        """Extrae el texto de una celda de la tabla de forma segura, manejando celdas
        vacías."""
        item = self.tabla_alumnos.item(row, col)
        # Si la celda es None (está vacía), retorna una cadena vacía en lugar de fallar
        al llamar .text()
        return item.text() if item is not None else ""

    def _mostrar_formulario(self, datos_alumno=None):
        """Función unificada para agregar o editar."""
        dialogo = DialogoAlumno(datos_alumno, self)

        if dialogo.exec_() == QDialog.Accepted:

            #DEBE DESEMPAQUETAR 4 VALORES
            matricula, nombre, contacto, email = dialogo.get_data()

            if datos_alumno is None:
                # Lógica Agregar
                resultado_mensaje = self.gestor.agregar_nuevo_alumno(matricula, nombre,
                contacto, email)
            else:
                # Lógica Editar (la matrícula ya está definida en el diálogo)
                resultado_mensaje = self.gestor.actualizar_datos_alumno(matricula,
                nombre, contacto, email)
            if "Error" in resultado_mensaje:
                QMessageBox.critical(self, "Error", resultado_mensaje)
            else:
                QMessageBox.information(self, "Operación Exitosa", resultado_mensaje)
                self._cargar_datos()

    def _mostrar_formulario_editar(self):
        """CORREGIDO: Prepara los datos de la fila seleccionada (U) manejando
        errores."""
        fila_seleccionada = self.tabla_alumnos.currentRow()
        if fila_seleccionada < 0:
            QMessageBox.warning(self, "Advertencia", "Por favor, seleccione un alumno
            para editar.")
            return

        datos_seleccionados = [
            self._get_cell_text_safe(fila_seleccionada, 0), # Matrícula
            self._get_cell_text_safe(fila_seleccionada, 1), # Nombre
            self._get_cell_text_safe(fila_seleccionada, 2), # Contacto
            self._get_cell_text_safe(fila_seleccionada, 3) # Email
        ]

```

```

        self._mostrar_formulario(datos_seleccionados)

    def _eliminar_alumno_seleccionado(self):
        fila_seleccionada = self.tabla_alumnos.currentRow()
        if fila_seleccionada < 0:
            QMessageBox.warning(self, "Advertencia", "Por favor, seleccione un alumno
para eliminar.")
            return
        matricula = self.tabla_alumnos.item(fila_seleccionada, 0).text()

        confirmacion = QMessageBox.question(self, "Confirmar Eliminación",
            f"¿Está seguro de que desea eliminar permanentemente al alumno con Matrícula
{matricula}? (BR.6)",
            QMessageBox.Yes | QMessageBox.No)

        if confirmacion == QMessageBox.Yes:
            resultado_mensaje = self.gestor.eliminar_alumno(matricula)

            if "Error" in resultado_mensaje:
                QMessageBox.critical(self, "Error de Eliminación", resultado_mensaje)
            else:
                QMessageBox.information(self, "Operación Exitosa", resultado_mensaje)
                self._cargar_datos()

    def _exportar_excel(self):
        """Exporta todos los datos del grupo a Excel."""
        # Solicitar directorio de destino
        directorio = QFileDialog.getExistingDirectory(self,
                                                    "Seleccionar carpeta para guardar
el archivo Excel",
                                                    "",
                                                    QFileDialog.ShowDirsOnly
                                                    )

        if not directorio:
            return

        # Mostrar mensaje de progreso
        QMessageBox.information(self,
                                "Exportando",
                                "Exportando datos a Excel. Puede tomar unos
segundos...")

        # Llamar al gestor de exportaciones
        exito, mensaje, ruta_archivo =
self._gestor_exportaciones.exportar_grupo_completo(
            self._grupo_id,
            self._nombre_grupo,
            directorio
        )

        if exito:
            QMessageBox.information(self, "Exportación Exitosa",
                                    f"{mensaje}\n\nArchivo guardado en:\n{ruta_archivo}")

```

```

        else:
            QMessageBox.critical(self, "Error en Exportación", mensaje)

# =====
# PUNTO DE ARRANQUE
# =====
if __name__ == '__main__':
    QApplication.setStyle(QStyleFactory.create('Fusion'))
    app = QApplication(sys.argv)

    try:
        with open('style.css', 'r') as f:
            app.setStyleSheet(f.read())
    except FileNotFoundError:
        print("Advertencia: El archivo style.css no fue encontrado.")

    ventana = VentanaAlumnos(grupo_id=1)
    ventana.show()
    sys.exit(app.exec_())

```

ventana [asistencia.py](#)

```

#presentacion/ventana_asistencia.py
import sys
from PyQt5.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QTableWidgetItem,
    QPushButton, QMessageBox, QTableWidgetItem, QHeaderView, QDateEdit,
    QComboBox, QLabel, QGroupBox, QGridLayout
)
from PyQt5.QtCore import QDate, Qt
from Logica.gestor_alumnos import GestorAsistencia
from datetime import date

class VentanaAsistencia(QWidget):
    """Ventana para el Caso de Uso 4: Registrar Asistencia."""

    def __init__(self, grupo_id, nombre_grupo):
        super().__init__()
        self._grupo_id = grupo_id
        self._nombre_grupo = nombre_grupo
        self.setWindowTitle(f"DirectAula - Asistencia {nombre_grupo}")
        self.resize(800, 400)
        self.gestor = GestorAsistencia(self._grupo_id)
        self._inicializar_ui(nombre_grupo)

    def _inicializar_ui(self, nombre_grupo):

        main_layout = QVBoxLayout()
        lbl_titulo = QLabel(f"Registro de Asistencia: {nombre_grupo}")
        lbl_titulo.setObjectName("titulo_principal")

```



```

main_layout.addWidget(lbl_titulo)

# -----
# SECCIÓN: CONTROL DE FECHA Y REGISTRO MASIVO
# -----

control_fecha_box = QGroupBox("Control de Asistencia")
control_layout = QGridLayout(control_fecha_box)

# 1. Selector de Fecha (QDateEdit)
lbl_fecha = QLabel("Seleccionar Fecha:")
self.fecha_asistencia = QDateEdit()
self.fecha_asistencia.setCalendarPopup(True) # Muestra el calendario al hacer
clic

self.fecha_asistencia.setDate(QDate.currentDate()) # Fecha de hoy por defecto
self.fecha_asistencia.dateChanged.connect(self._cargar_datos)
control_layout.addWidget(lbl_fecha, 0, 0)
control_layout.addWidget(self.fecha_asistencia, 0, 1)

# 2. Botón de Registro Masivo
self.btn_masivo = QPushButton("Poner Asistencia a todos")
self.btn_masivo.setObjectName("btn_agregar")
self.btn_masivo.clicked.connect(self._registrar_asistencia_masiva)

control_layout.addWidget(self.btn_masivo, 1, 0, 1, 2) # Ocupa 2 columnas
main_layout.addWidget(control_fecha_box)

# -----
# TABLA DE ASISTENCIA
# -----

self.tabla_asistencia = QTableWidget()
self.tabla_asistencia.setColumnCount(3)
self.tabla_asistencia.setHorizontalHeaderLabels(["Matrícula", "Nombre Completo",
"Estado"])

self.tabla_asistencia.horizontalHeader().setSectionResizeMode(QHeaderView.Stretch)
main_layout.addWidget(self.tabla_asistencia)
self.setLayout(main_layout)
self._cargar_datos() # Carga los datos al iniciar con la fecha de hoy

def _cargar_datos(self):
    """Muestra los datos de asistencia del grupo para la fecha seleccionada."""
    # 1. Obtener la fecha seleccionada
    fecha = self.fecha_asistencia.date().toString("yyyy-MM-dd")

    # 2. Obtener los datos del gestor
    datos = self.gestor.obtener_asistencia_para_ui(fecha)

    # 3. Configurar la tabla
    self.tabla_asistencia.setRowCount(len(datos))

    # 4. Iterar y llenar la tabla
    for fila_indice, alumno_data in enumerate(datos):

        # Columna 0: Matrícula (Solo Lectura)

```

```

        item_matricula = QTableWidgetItem(alumno_data[0])
        item_matricula.setFlags(item_matricula.flags() & ~Qt.ItemIsEditable) #
Desactivar edición
        self.tabla_asistencia.setItem(fila_indice, 0, item_matricula)

        # Columna 1: Nombre Completo (Solo Lectura)
        item_nombre = QTableWidgetItem(alumno_data[1])
        item_nombre.setFlags(item_nombre.flags() & ~Qt.ItemIsEditable) # Desactivar
edición
        self.tabla_asistencia.setItem(fila_indice, 1, item_nombre)

        # Columna 2: Estado (QComboBox para interactividad)
        estado_combo = QComboBox()
        estados = ["Presente", "Ausente", "Retardo", "Justificado"]
        estado_combo.addItems(estados)

        # Seleccionar el estado actual (devuelto por el DAO: 'Presente', 'Ausente',
etc.)
        estado_combo.setCurrentText(alumno_data[2])

        # lambda para capturar los valores específicos de esta fila (matrícula y
fecha)
        estado_combo.currentIndexChanged.connect(
            lambda index, m=alumno_data[0], d=fecha, combo=estado_combo:
                self._actualizar_asistencia_individual(m, d, combo.currentText())
        )

        self.tabla_asistencia.setCellWidget(fila_indice, 2, estado_combo)

def _registrar_asistencia_masiva(self):
    """Llama al BLL para marcar a todos como Presente y recarga la tabla."""
    fecha = self.fecha_asistencia.date().toString("yyyy-MM-dd")

    confirmacion = QMessageBox.question(self, "Confirmar Registro",
        f"¿Desea marcar a TODOS los alumnos como 'Asistencia' para la fecha
{fecha}?",
        QMessageBox.Yes | QMessageBox.No)

    if confirmacion == QMessageBox.Yes:
        resultado_mensaje = self.gestor.registrar_asistencia_masiva(fecha)

        if "Error" in resultado_mensaje:
            QMessageBox.critical(self, "Error", resultado_mensaje)
        else:
            QMessageBox.information(self, "Operación Exitosa", resultado_mensaje)
            self._cargar_datos()

def _actualizar_asistencia_individual(self, matricula, fecha, nuevo_estado):
    """Guarda el estado de un alumno individualmente (Activado por el ComboBox)."""
    resultado_mensaje = self.gestor.actualizar_estado_asistencia(matricula, fecha,
nuevo_estado)

```

```
if "Error" in resultado_mensaje:
    QMessageBox.critical(self, "Error de Guardado", resultado_mensaje)
```

ventana_calificacion_final.py

```
#Archivo para la ventana de calificación final y estado de riesgo.
from PyQt5.QtWidgets import (
    QDialog, QVBoxLayout, QLabel,
    QPushButton, QTableWidgetItem, QHeaderView,
    QMessageBox, QFileDialog
)
from PyQt5.QtCore import Qt
from Logica.gestor_alumnos import GestorCalificaciones
from Logica.gestor_exportacion import GestorExportaciones
#Clase para la ventana de calificación final y estado de riesgo.
class VentanaCalificacionFinal(QDialog):
    #Método constructor.
    def __init__(self, grupo_id, nombre_grupo, parent=None):
        super().__init__(parent)
        self._grupo_id = grupo_id
        self._nombre_grupo = nombre_grupo

        self.setWindowTitle(f"Calificación final - {nombre_grupo} ")
        self.setGeometry(100, 100, 800, 400)

        self._setup_ui()
        self._cargar_resultados()
#Método para configurar la interfaz de usuario.
    def _setup_ui(self):
        layout_principal = QVBoxLayout(self)

        #Etiqueta de título que muestra el grupo actual
        lbl_titulo = QLabel(f"Resultados finales del grupo:
{self._nombre_grupo}")
        lbl_titulo.setObjectName("titulo_principal")
        lbl_titulo.setAlignment(Qt.AlignCenter)
        layout_principal.addWidget(lbl_titulo)

        #2. Tabla de Resultados
        self.tabla_resultados = QTableWidgetItem()
```

```

        self.tabla_resultados.setColumnCount(5)
        self.tabla_resultados.setHorizontalHeaderLabels([
            "Matrícula", "Nombre Completo", "Promedio Final",
            "Asistencia (%)", "Estado de Riesgo"
        ])

        #Configuración para que las columnas se ajusten
        automáticamente

        self.tabla_resultados.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeToContents)

        self.tabla_resultados.horizontalHeader().setStretchLastSection(True)

        layout_principal.addWidget(self.tabla_resultados)

        btn_reporte = QPushButton("Generar Reporte Excel")
        btn_reporte.setObjectName("btn_exportar")
        btn_reporte.clicked.connect(self._generar_reporte_excel)
        layout_principal.addWidget(btn_reporte)

    #Método para cargar los resultados finales en la tabla.
    def _cargar_resultados(self):

        try:
            #Usamos el grupo_id que ya recibimos en el constructor
            gestor_calif = GestorCalificaciones(self._grupo_id)

            #Llamada al método central de la lógica (CU6)
            resultados =
gestor_calif.calcular_promedios_y_estado_final()

            self.tabla_resultados.setRowCount(len(resultados))
            #Llenar la tabla con los datos obtenidos
            for row, data in enumerate(resultados):
                matricula, nombre, promedio, asistencia, riesgo =
data

                self.tabla_resultados.setItem(row, 0,
QTableWidgetItem(matricula))
                self.tabla_resultados.setItem(row, 1,
QTableWidgetItem(nombre))

```

```

        #Promedio final, redondeo a 2 decimales
        item_promedio = QTableWidgetItem(f"{promedio:.2f}")
        item_promedio.setTextAlignment(Qt.AlignCenter)
        self.tabla_resultados.setItem(row, 2, item_promedio)

        #Asistencia %
        item_asistencia =
QTableWidgetItem(f"{asistencia:.2f}%")
        item_asistencia.setTextAlignment(Qt.AlignCenter)
        self.tabla_resultados.setItem(row, 3,
item_asistencia)

        #Estado de riesgo
        item_riesgo = QTableWidgetItem(riesgo)
        item_riesgo.setTextAlignment(Qt.AlignCenter)

        #Resaltado visual para el estado de riesgo
        if "Riesgo" in riesgo:
            #Color rojo
            item_riesgo.setForeground(Qt.red)
        elif riesgo == "Normal":
            # Color verde para estado Normal
            item_riesgo.setForeground(Qt.darkGreen)

        self.tabla_resultados.setItem(row, 4, item_riesgo)
        #Ajustar el tamaño de las filas
    except Exception as e:
        QMessageBox.critical(self, "Error de Carga", f"Ocurrió un
error al cargar los datos: {str(e)}")
        #Método para generar reporte Excel
        def _generar_reporte_excel(self):

            directorio = QFileDialog.getExistingDirectory(
                self, "Seleccionar carpeta para guardar el reporte",
                "",
                QFileDialog.ShowDirsOnly
            )
            if not directorio:
                return
            QMessageBox.information(self,"Generando reporte",
                "Generando reporte, puede tomar unos

```

```
segundos.")

    gestor_export = GestorExportaciones()
    exito, mensaje, ruta_archivo =
gestor_export.exportar_grupo_completo(
        self._grupo_id,
        self._nombre_grupo,
        directorio
    )
    if exito:
        QMessageBox.information(self, "Reporte generado",
                                f"{mensaje}\n\nReporte guardado
en:\n{ruta_archivo}")
    else:
        QMessageBox.critical(self, "Error ", mensaje)
```

ventana_calificaciones_menu.py

```
#Archivo para la ventana de menú de calificaciones.
from PyQt5.QtWidgets import QWidget, QVBoxLayout, QLabel,
QPushButton, QMessageBox
from PyQt5.QtCore import Qt
from Presentacion.ventana_ponderacion import VentanaPonderacion
from Presentacion.ventana_registro_calificaciones import
VentanaRegistroCalificaciones
from Presentacion.ventana_calificacion_final import
VentanaCalificacionFinal

#Clase para la ventana de menú de calificaciones.
class VentanaCalificacionesMenu(QWidget):
    #Método constructor.
    def __init__(self, grupo_id, nombre_grupo, parent=None):
        super().__init__(parent)
        self._grupo_id = grupo_id
        self._nombre_grupo = nombre_grupo
        self.setWindowTitle(f"Calificaciones - {nombre_grupo}")
        self.resize(500, 300)
        self._inicializar_ui()
        #Método para inicializar la interfaz de usuario.
    def _inicializar_ui(self):
        layout = QVBoxLayout(self)
```

```

        #Título principal
        lbl_titulo = QLabel(f"Gestión de Calificaciones:
{self._nombre_grupo}")
        lbl_titulo.setObjectName("titulo_principal")
        lbl_titulo.setAlignment(Qt.AlignCenter)
        layout.addWidget(lbl_titulo)

        #CU3: Administrar Ponderación
        btn_ponderacion = QPushButton("1. Administrar Ponderación")
        btn_ponderacion.setObjectName("btn_agregar")
        btn_ponderacion.clicked.connect(self.abrir_ponderacion)
        layout.addWidget(btn_ponderacion)

        #CU5: Registrar Calificaciones
        btn_registro = QPushButton("2. Registrar Calificaciones")
        btn_registro.setObjectName("btn_agregar")
        btn_registro.clicked.connect(self.abrir_registro)
        layout.addWidget(btn_registro)

        #Ver Calificación Final Ponderada y Estado de Riesgo
        btn_final = QPushButton("3. Calificación Final y Estado de
Riesgo")
        btn_final.setObjectName("btn_agregar")
        btn_final.clicked.connect(self.abrir_calificacion_final)
        layout.addWidget(btn_final)

        #Establecer el diseño principal
        def abrir_ponderacion(self):
            self.ventana_ponderacion = VentanaPonderacion(self._grupo_id,
self._nombre_grupo) #Abrir ventana de ponderación
            self.ventana_ponderacion.show()
#Método para abrir la ventana de registro de calificaciones.
        def abrir_registro(self):
            self.ventana_registro =
VentanaRegistroCalificaciones(self._grupo_id, self._nombre_grupo)
            self.ventana_registro.show()
        #Método para abrir la ventana de calificación final y estado de
riesgo.
        def abrir_calificacion_final(self):
            self.ventana_final = VentanaCalificacionFinal(self._grupo_id,
self._nombre_grupo, self)
            self.ventana_final.exec_()

```

ventana_grupos.py

```
#Archivo para la gestión de la ventana de grupos.
from PyQt5.QtWidgets import (
    QWidget, QVBoxLayout, QHBoxLayout, QLineEdit, QTableWidgetItem,
    QPushButton, QMessageBox, QDialog, QFormLayout, QDialogButtonBox,
    QLabel, QTableWidgetItem, QHeaderView
)
from Logica.gestor_alumnos import GestorGrupos

#Clase para agregar o editar un grupo.
class DialogoGrupo(QDialog):
    #Método constructor.
    def __init__(self, datos_grupo=None, parent=None): #Ningún dato
al inicio.
        super().__init__(parent) #Llama al constructor padre.
        self.datos_grupo = datos_grupo
        self.setWindowTitle("Registrar Grupo" if not datos_grupo else
"Editar Grupo")
        layout = QFormLayout()
        self.campo_nombre = QLineEdit()
        self.campo_ciclo = QLineEdit()
        #Si se proporcionan datos para editar, se llenan los campos.
        if datos_grupo:
            #datos_grupo = [id, nombre, ciclo]
            self.campo_nombre.setText(datos_grupo[1])
            self.campo_ciclo.setText(datos_grupo[2])

        layout.addRow(QLabel("Nombre del Grupo *"),
self.campo_nombre)
        layout.addRow(QLabel("Ciclo Escolar *"), self.campo_ciclo)
#Botones Aceptar y Cancelar.
        self.botones = QDialogButtonBox(QDialogButtonBox.Ok |
QDialogButtonBox.Cancel, self)
        self.botones.accepted.connect(self.accept)
        self.botones.rejected.connect(self.reject)

        layout.addRow(self.botones)
        self.setLayout(layout)
```



```

#Método para obtener los datos ingresados.
def get_data(self):
    grupo_id = self.datos_grupo[0] if self.datos_grupo else None

    return (
        grupo_id,
        self.campo_nombre.text().strip(),
        self.campo_ciclo.text().strip()
    )

#Clase principal de la ventana de grupos.
class VentanaGrupos(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("DirectAula - Administrar Grupos")
        self.resize(600, 400)
        self.gestor = GestorGrupos()
        self._inicializar_ui()
        self._cargar_datos()

#Inicialización de la interfaz de usuario.
def _inicializar_ui(self):
    main_layout = QVBoxLayout()

    #TÍTULO PRINCIPAL
    lbl_titulo = QLabel("DirectAula - Administración de Grupos")
    lbl_titulo.setObjectName("titulo_principal")
    main_layout.addWidget(lbl_titulo)

    #Botones
    top_bar_layout = QHBoxLayout()
    self.btn_agregar = QPushButton("+ Agregar Grupo")
    self.btn_agregar.setObjectName("btn_agregar")
    self.btn_agregar.clicked.connect(lambda:
self._mostrar_formulario(None))

    self.btn_editar = QPushButton("✎ Editar Grupo")
    self.btn_editar.setObjectName("btn_editar")

self.btn_editar.clicked.connect(self._mostrar_formulario_editar)

    self.btn_eliminar = QPushButton("🗑 Eliminar Grupo")
    self.btn_eliminar.setObjectName("btn_eliminar")

```

```

self.btn_eliminar.clicked.connect(self._eliminar_grupo_seleccionado)
    #Añadir botones al layout superior
    top_bar_layout.addWidget(self.btn_agregar)
    top_bar_layout.addWidget(self.btn_editar)
    top_bar_layout.addWidget(self.btn_eliminar)
#Barra superior con botones
    main_layout.addLayout(top_bar_layout)

    #Tabla de grupos
    lbl_subtitulo = QLabel("Lista de Grupos Registrados")
    lbl_subtitulo.setProperty("class", "subtitulo")
    main_layout.addWidget(lbl_subtitulo)
    #Tabla de grupos
    self.tabla_grupos = QTableWidget()
    self.tabla_grupos.setColumnCount(3)
    self.tabla_grupos.setHorizontalHeaderLabels(["ID", "Nombre
del Grupo", "Ciclo Escolar"])

self.tabla_grupos.horizontalHeader().setSectionResizeMode(QHeaderView
.Stretch)

    self.tabla_grupos.setColumnHidden(0, True) #Ocultar la
columna ID
    main_layout.addWidget(self.tabla_grupos)

    self.setLayout(main_layout)

#Carga de datos en la tabla
    def _cargar_datos(self):
        datos = self.gestor.obtener_lista_grupos() # [id, nombre,
ciclo]

        self.tabla_grupos.setRowCount(0)
        #Insertar filas con los datos obtenidos
        for fila_indice, grupo_data in enumerate(datos):
            self.tabla_grupos.insertRow(fila_indice)

            #Insertamos las 3 columnas (ID, Nombre, Ciclo Escolar)
            for columna, valor in enumerate(grupo_data):
                item = QTableWidgetItem(str(valor))
                self.tabla_grupos.setItem(fila_indice, columna, item)

```

```

#Mostrar formulario para agregar o editar grupo
def _mostrar_formulario(self, datos_grupo=None):
    dialogo = DialogoGrupo(datos_grupo, self)
    if dialogo.exec_() == QDialog.Accepted:

        grupo_id, nombre, ciclo = dialogo.get_data()

        if grupo_id is None:
            #Lógica Agregar
            resultado_mensaje =
self.gestor.agregar_nuevo_grupo(nombre, ciclo)
        else:
            #Lógica Editar
            resultado_mensaje =
self.gestor.actualizar_datos_grupo(grupo_id, nombre, ciclo)

        if "Error" in resultado_mensaje:
            QMessageBox.critical(self, "Error",
resultado_mensaje)
        else:
            QMessageBox.information(self, "Operación Exitosa",
resultado_mensaje)
            self._cargar_datos()

def _mostrar_formulario_editar(self):
    #Prepara los datos de la fila seleccionada
    fila_seleccionada = self.tabla_grupos.currentRow()
    if fila_seleccionada < 0:
        QMessageBox.warning(self, "Advertencia", "Por favor,
seleccione un grupo para editar.")
        return

    #Obtenemos ID, Nombre y Ciclo
    datos_seleccionados = [
        int(self.tabla_grupos.item(fila_seleccionada, 0).text()),
#ID (Oculto)
        self.tabla_grupos.item(fila_seleccionada, 1).text(),
#Nombre
        self.tabla_grupos.item(fila_seleccionada, 2).text()
#Ciclo
    ]

    self._mostrar_formulario(datos_seleccionados)

```

```

#Método para eliminar el grupo seleccionado
def _eliminar_grupo_seleccionado(self):
    fila_seleccionada = self.tabla_grupos.currentRow()
    if fila_seleccionada < 0:
        QMessageBox.warning(self, "Advertencia", "Por favor,
seleccione un grupo para eliminar.")
        return

    grupo_id = int(self.tabla_grupos.item(fila_seleccionada,
0).text())
    nombre_grupo = self.tabla_grupos.item(fila_seleccionada,
1).text()

    confirmacion = QMessageBox.question(self, "Confirmar
Eliminación",
        f"¿Está seguro de que desea eliminar permanentemente el
grupo '{nombre_grupo}'? (BR.2)",
        QMessageBox.Yes | QMessageBox.No)

    if confirmacion == QMessageBox.Yes:
        resultado_mensaje = self.gestor.eliminar_grupo(grupo_id)

        if "Error" in resultado_mensaje:
            QMessageBox.critical(self, "Error de Eliminación",
resultado_mensaje)
        else:
            QMessageBox.information(self, "Operación Exitosa",
resultado_mensaje)
            self._cargar_datos()

```

ventana_login.py

```

# presentacion/ventana_login.py
import sys
from PyQt5.QtWidgets import (
    QApplication, QWidget, QVBoxLayout, QLineEdit,
    QPushButton, QLabel, QMessageBox, QFrame, QDialog, QHBoxLayout, QSpacerItem,
    QSizePolicy
)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from Logica.gestor_autenticacion import GestorAutenticacion

```

```

from Presentacion.ventana_registro_profesor import VentanaRegistroProfesor

class VentanaLogin(QWidget):
    def __init__(self):
        super().__init__()
        # Inicializa el gestor de autenticación
        self.gestor = GestorAutenticacion()
        self.ventana_principal = None
        self._inicializar_ui()

    def _inicializar_ui(self):
        self.setWindowTitle("DirectAula - Iniciar Sesión")
        self.setFixedSize(450, 450)
        self.setStyleSheet("""
            VentanaLogin {
                background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
                    stop:0 #EAF6FF,
                    stop:0.5 #F6FBFF,
                    stop:1 #EAFBF2);
            }
            QLineEdit {
                border: 1px solid #D6E9F8;
                border-radius: 8px;
                padding: 10px;
                font-size: 14px;
                background: #FF9999;
                color: #2B3A42;
            }
            QLineEdit:focus {
                border: 2px solid #7FB7FF;
                background: #FFFFFF;
            }
            QPushButton {
                background-color: #5AA6FF;
                color: white;
                border: none;
                border-radius: 8px;
                font-size: 16px;
                font-weight: bold;
                padding: 10px;
            }
            QPushButton:hover {
                background-color: #3C8EE6;
            }
            QPushButton:disabled {
                background-color: #BFDFFF;
                color: #7A8B96;
            }
            QLabel {
                color: #334E5A;
            }
        """)

```

```

# Estilo de fondo con degradado
self.setStyleSheet("""
    VentanaLogin {
        background: qlineargradient(x1:0, y1:0, x2:1, y2:1,
            stop:0 #DFF3FF,      /* azul pastel */
            stop:0.5 #E8FFF5,    /* verde pastel */
            stop:1 #FFE6E6);     /* rojo pastel suave */
    }

    QLineEdit {
        border: 1px solid #BBD7E4;
        border-radius: 8px;
        padding: 10px;
        font-size: 14px;
        background: #F0F8FF;     /* azul muy suave */
        color: #2B3A42;
    }

    QLineEdit:focus {
        border: 2px solid #7FB7FF;
        background: #FFFFFF;
    }

    QPushButton {
        background-color: #7EC8E3; /* azul claro suave */
        color: white;
        border: none;
        border-radius: 8px;
        font-size: 16px;
        font-weight: bold;
        padding: 10px;
    }

    QPushButton:hover {
        background-color: #68B3D1; /* azul suave más oscuro */
    }

    QPushButton:disabled {
        background-color: #CFEAF5;
        color: #7A8B96;
    }

    QLabel {
        color: #334E5A;
    }
""")

main_layout = QVBoxLayout(self)
main_layout.setAlignment(Qt.AlignCenter)
main_layout.setSpacing(0)
main_layout.setContentsMargins(30, 30, 30, 30)

```

```

# Frame principal
main_frame = QFrame()
main_frame.setFixedSize(380, 480)
main_frame.setStyleSheet("""
    QFrame {
        background-color: rgba(255, 255, 255, 0.95);
        border-radius: 15px;
        box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
    }
""")

frame_layout = QVBoxLayout(main_frame)
frame_layout.setAlignment(Qt.AlignTop)
frame_layout.setSpacing(15)
frame_layout.setContentsMargins(30, 30, 30, 30)

# --- Logo y Título ---

# Logo/Icono
lbl_icono = QLabel("👤")
lbl_icono.setAlignment(Qt.AlignCenter)
lbl_icono.setStyleSheet("font-size: 60px; color: #FF6B35;")
lbl_icono.setFixedHeight(80)
frame_layout.addWidget(lbl_icono)

# Título
lbl_titulo = QLabel("Bienvenido a DirectAula")
lbl_titulo.setAlignment(Qt.AlignCenter)
lbl_titulo.setFont(QFont("Arial", 18, QFont.Bold))
lbl_titulo.setStyleSheet("color: #2D3748; margin-bottom: 5px;")
frame_layout.addWidget(lbl_titulo)

# Subtítulo
lbl_subtitulo = QLabel("Ingresa tus credenciales de Profesor")
lbl_subtitulo.setAlignment(Qt.AlignCenter)
lbl_subtitulo.setFont(QFont("Arial", 10))
lbl_subtitulo.setStyleSheet("color: #718096; margin-bottom: 20px;")
frame_layout.addWidget(lbl_subtitulo)

# --- Formulario de login ---

self.campo_usuario = QLineEdit()
self.campo_usuario.setPlaceholderText("Nombre de usuario")
self.campo_usuario.setFixedHeight(45)
frame_layout.addWidget(self.campo_usuario)

self.campo_password = QLineEdit()
self.campo_password.setPlaceholderText("Contraseña")
self.campo_password.setEchoMode(QLineEdit.Password)
self.campo_password.setFixedHeight(45)
frame_layout.addWidget(self.campo_password)

```

```

        frame_layout.addSpacing(15)

        # Botón de login
        btn_login = QPushButton("Iniciar Sesión")
        btn_login.setFixedHeight(45)
        btn_login.clicked.connect(self._autenticar_usuario)
        frame_layout.addWidget(btn_login)

        frame_layout.addSpacing(10)

        # Botón de registro
        btn_registro = QPushButton("Crear cuenta de Profesor")
        btn_registro.setStyleSheet("border: none; color: #4299E1; font-size: 13px;
padding: 5px;")
        btn_registro.clicked.connect(self._abrir_registro)
        frame_layout.addWidget(btn_registro)

        # Espaciador para centrar verticalmente el contenido
        frame_layout.addItem(QSpacerItem(20, 10, QSizePolicy.Minimum,
QSizePolicy.Expanding))

        # Texto informativo (demo)
        lbl_info = QLabel("")
        lbl_info.setAlignment(Qt.AlignCenter)
        lbl_info.setStyleSheet("""
            QLabel {
                color: #718096; font-size: 10px; font-style: italic;
                background-color: rgba(237, 242, 247, 0.8);
                padding: 6px 10px; border-radius: 5px;
            }
        """)
        lbl_info.setFixedHeight(30)
        frame_layout.addWidget(lbl_info)

        main_layout.addWidget(main_frame)

        # Conectar eventos de teclado (Enter)
        self.campo_password.returnPressed.connect(btn_login.click)
        self.campo_usuario.setFocus()

        # =====
        # MÉTODOS DE LA LÓGICA DE LOGIN (CORREGIDOS Y AÑADIDOS)
        # =====

        def _autenticar_usuario(self):
            """
            [CORREGIDO] Autentica al usuario usando GestorAutenticacion.
            Este era el método faltante que causaba el AttributeError.
            """

            usuario = self.campo_usuario.text().strip()
            password = self.campo_password.text().strip()

            if not usuario or not password:
                QMessageBox.warning(self, "Error", "Por favor ingrese usuario y contraseña")

```



```

        self.campo_usuario.setFocus()
        return

    # Deshabilitar interfaz durante autenticación
    self.setEnabled(False)
    QApplication.processEvents()

    try:
        resultado = self.gestor.autenticar_profesor(usuario, password)

        if resultado["autenticado"]:
            QMessageBox.information(self, "✅ Éxito", f"Bienvenido, {resultado['nombre']}!")
            self._abrir_ventana_principal()
        else:
            QMessageBox.critical(self, "Error de Autenticación",
            resultado["mensaje"])
            self.campo_password.selectAll()
            self.campo_password.setFocus()

    except Exception as e:
        # Captura errores de base de datos o lógica
        QMessageBox.critical(self, "Error del Sistema", f"Error al conectar con el
sistema: {str(e)}")
        self.campo_usuario.setFocus()
    finally:
        self.setEnabled(True)

    def _abrir_ventana_principal(self):
        """
        [CORREGIDO] Cierra la ventana de login y abre la ventana principal.
        """
        try:
            # Importación local para evitar dependencia circular
            from app import VentanaMenuPrincipal
            self.ventana_principal = VentanaMenuPrincipal()
            self.ventana_principal.show()
            self.close()
        except ImportError:
            QMessageBox.critical(self, "Error", "No se pudo cargar la Ventana Menu
Principal. Verifique app.py.")
        except Exception as e:
            QMessageBox.critical(self, "Error", f"No se pudo abrir la ventana principal:
{str(e)}")

    def _abrir_registro(self):
        """
        [CORREGIDO] Abre la ventana de registro de profesor como modal.
        """
        ventana_registro = VentanaRegistroProfesor(self)
        if ventana_registro.exec_() == QDialog.Accepted:
            # Si el registro fue exitoso, llenamos el campo de usuario

```

```

        self.campo_usuario.setText(ventana_registro.get_usuario_registrado())
        self.campo_password.setFocus()

if __name__ == "__main__":
    app = QApplication(sys.argv)
    ventana_login = VentanaLogin()
    ventana_login.show()
    sys.exit(app.exec_())

```

ventana_ponderacion.py

```

#Archivo para la ventana de ponderación de categorías.
from PyQt5.QtWidgets import (
    QWidget, QVBoxLayout, QWidget, QHBoxLayout,
    QLabel, QPushButton, QMessageBox, QTableWidgetItem, QHeaderView,
    QLineEdit
)
from PyQt5.QtCore import Qt
from Logica.gestor_alumnos import GestorCalificaciones

#Clase para la ventana de ponderación de categorías.
class VentanaPonderacion(QWidget):
    #Método constructor.
    #Atributos privados: grupo_id, nombre_grupo, gestor
    def __init__(self, grupo_id, nombre_grupo, parent=None):
        super().__init__(parent)
        self._grupo_id = grupo_id
        self._nombre_grupo = nombre_grupo
        self.gestor = GestorCalificaciones(grupo_id)
        self.setWindowTitle(f"Ponderación - {nombre_grupo}")
        self.resize(800, 450)
        self._inicializar_ui()
        self._cargar_datos()

#Método para inicializar la interfaz de usuario.
    def _inicializar_ui(self):
        main_layout = QVBoxLayout(self)

        lbl_titulo = QLabel(f"Definir Categorías y Ponderación:
{self._nombre_grupo}")
        lbl_titulo.setObjectName("titulo_principal")
        main_layout.addWidget(lbl_titulo)

```

```

# Tabla para categorías dinámicas
self.tabla_ponderacion = QTableWidgetItem()
self.tabla_ponderacion.setColumnCount(3)
self.tabla_ponderacion.setHorizontalHeaderLabels([
    "Categoría",
    "Valor (%)",
    "Número tareas/participaciones"
])
#Ajustar ancho de las columnas

self.tabla_ponderacion.horizontalHeader().setSectionResizeMode(0,
QHeaderView.Stretch)

self.tabla_ponderacion.horizontalHeader().setSectionResizeMode(1,
QHeaderView.ResizeToContents)

self.tabla_ponderacion.horizontalHeader().setSectionResizeMode(2,
QHeaderView.ResizeToContents)

#Conexión para actualizar la suma visualmente al cambiar
cualquier celda de peso

self.tabla_ponderacion.cellChanged.connect(self._actualizar_suma)
main_layout.addWidget(self.tabla_ponderacion)

#Indicador visual de la suma actual (FE.1)
self.lbl_suma = QLabel("Suma Actual: 0.0%")
self.lbl_suma.setObjectName("subtitulo")
main_layout.addWidget(self.lbl_suma)

#Botones de acción
btn_layout = QHBoxLayout()

btn_agregar = QPushButton("Añadir Categoría")
btn_agregar.clicked.connect(self._agregar_fila)
btn_layout.addWidget(btn_agregar)

btn_eliminar = QPushButton("Eliminar Categoría Seleccionada")
btn_eliminar.setObjectName("btn_eliminar")
btn_eliminar.clicked.connect(self._eliminar_fila)
btn_layout.addWidget(btn_eliminar)

```

```

        btn_guardar = QPushButton("Guardar Estructura")
        btn_guardar.setObjectName("btn_agregar")
        btn_guardar.clicked.connect(self._guardar_ponderacion)
        btn_layout.addWidget(btn_guardar)

    main_layout.addLayout(btn_layout)
    self.setLayout(main_layout)

#Método para cargar los datos existentes en la tabla.
    def _cargar_datos(self):
        #1. Obtener las categorías desde el gestor
        categorias = self.gestor.obtener_categorias_evaluacion()
        self.tabla_ponderacion.setRowCount(len(categorias))

        self.tabla_ponderacion.blockSignals(True)
        for fila_indice, cat in enumerate(categorias):
            #Columna 0: Nombre
            self.tabla_ponderacion.setItem(fila_indice, 0,
            QTableWidgetItem(cat.get_nombre_categoria()))
            #Columna 1: Peso
            self.tabla_ponderacion.setItem(fila_indice, 1,
            QTableWidgetItem(str(cat.get_peso_porcentual())))
            #Columna 2: Max Items
            self.tabla_ponderacion.setItem(fila_indice, 2,
            QTableWidgetItem(str(cat.get_max_items())))

        self.tabla_ponderacion.blockSignals(False)
        self._actualizar_suma()

#Método para añadir una nueva fila.
    def _agregar_fila(self):

        row_count = self.tabla_ponderacion.rowCount()
        self.tabla_ponderacion.insertRow(row_count)
        self.tabla_ponderacion.setItem(row_count, 1,
        QTableWidgetItem("0.0"))
        self.tabla_ponderacion.setItem(row_count, 2,
        QTableWidgetItem("1"))

#Método para eliminar la fila seleccionada.
    def _eliminar_fila(self):
        #Obtener la fila seleccionada

```

```

        fila_seleccionada = self.tabla_ponderacion.currentRow()
        if fila_seleccionada >= 0:
            self.tabla_ponderacion.removeRow(fila_seleccionada)
            self._actualizar_suma()

#Método para actualizar la suma de los pesos.
    def _actualizar_suma(self):
        #Calcular la suma de los pesos ingresados
        suma = 0.0
        try:
            for i in range(self.tabla_ponderacion.rowCount()):
                item = self.tabla_ponderacion.item(i, 1)
                if item and item.text():
                    suma += float(item.text())

            #Actualizar la etiqueta visual
            self.lbl_suma.setText(f"Suma Actual: {suma:.1f}%")
            if round(suma) != 100:
                self.lbl_suma.setStyleSheet("color: red; font-weight:
bold;")
            else:
                self.lbl_suma.setStyleSheet("color: green;
font-weight: bold;")
        except ValueError:
            self.lbl_suma.setText("Suma Actual: Inválida (Valores no
numéricos en Peso)")
            self.lbl_suma.setStyleSheet("color: red; font-weight:
bold;")

#Método para obtener los datos de la tabla.
    def _obtener_datos_tabla(self):
        datos = []
        for i in range(self.tabla_ponderacion.rowCount()):
            nombre_item = self.tabla_ponderacion.item(i, 0)
            peso_item = self.tabla_ponderacion.item(i, 1)
            max_item = self.tabla_ponderacion.item(i, 2) #Número de
tareas/participaciones

            #Validar que no haya campos vacíos
            if not (nombre_item and peso_item and max_item and
nombre_item.text().strip()):
                QMessageBox.critical(self, "Error de Datos", f"La
fila {i+1} tiene campos vacíos o la Categoría no tiene nombre.")

```

```

        return None

    try:
        nombre = nombre_item.text().strip()
        peso = float(peso_item.text())
        max_items = int(max_item.text())
        datos.append((nombre, peso, max_items))
    except ValueError:
        QMessageBox.critical(self, "Error de Datos",
f"Asegúrese de que los números de la fila {i+1} son números válidos.")

        return None
    return datos

#Método para guardar la ponderación definida.
def _guardar_ponderacion(self):

    datos_a_guardar = self._obtener_datos_tabla()
    if datos_a_guardar is None:
        return

    # FE.2: Modificación con promedios ya calculados
    alerta = QMessageBox.question(self, "Recálculo de Promedios
(FE.2)",

        "Guardar esta nueva estructura ELIMINARÁ la anterior y
forzará el recálculo de todos los promedios. ¿Desea continuar?",
        QMessageBox.Yes | QMessageBox.Cancel
    )

    if alerta == QMessageBox.Cancel:
        return

    #6. Guardar en la base de datos mediante el gestor logico
    resultado =
self.gestor.guardar_categorias_evaluacion(datos_a_guardar)

    if "Error" in resultado:
        QMessageBox.critical(self, "Error al Guardar", resultado)
    else:
        QMessageBox.information(self, "Éxito", resultado)
        self.close()

```

ventana_registro_calificaciones.py

```
#Archivo para la ventana de registro de calificaciones.
from PyQt5.QtWidgets import (
    QWidget, QVBoxLayout, QTableWidget, QComboBox,
    QLabel, QPushButton, QMessageBox, QTableWidgetItem, QHeaderView
)
from PyQt5.QtCore import Qt
from Logica.gestor_alumnos import GestorCalificaciones
#Clase para la ventana de registro de calificaciones.
class VentanaRegistroCalificaciones(QWidget):
    #Método constructor
    def __init__(self, grupo_id, nombre_grupo, parent=None):
        super().__init__(parent)
        self._grupo_id = grupo_id
        self._nombre_grupo = nombre_grupo
        # Inicializa el gestor de calificaciones
        self.gestor = GestorCalificaciones(grupo_id)
        self.setWindowTitle(f"Registro de calificaciones -
{nombre_grupo}")
        self.resize(800, 400)

        #Llama a _obtener_nombres_categorias antes de _inicializar_ui
        self._categorias_activas = self._obtener_nombres_categorias()

        self._inicializar_ui()
    #Método para obtener los nombres de las categorías definidas
    def _obtener_nombres_categorias(self):
        categorias = self.gestor.obtener_categorias_evaluacion()
        nombres = [c.get_nombre_categoria() for c in categorias]

        #El gestor de calificaciones debe retornar los objetos
actualizados con la ponderación definida por el maestro.
        return nombres
    #Método para inicializar la interfaz de usuario.
    def _inicializar_ui(self):
        main_layout = QVBoxLayout(self)
        #Título principal
        lbl_titulo = QLabel(f"Registrar calificaciones:
{self._nombre_grupo}")
        lbl_titulo.setObjectName("titulo_principal")
        main_layout.addWidget(lbl_titulo)
```

```

#1. Selector de Categoría
selector_layout = QVBoxLayout()
lbl_categoria = QLabel("Seleccionar categoría:")
self.combo_categoria = QComboBox()
#Manejo de caso sin categorías definidas
if not self._categorias_activas:
    self.combo_categoria.addItem("No hay categorías
definidas")
    QMessageBox.warning(self, "Advertencia", "No hay
categorías de evaluación definidas para este grupo. Vaya a
Administrar Ponderación (CU3).")
    self.btn_guardar = QPushButton("Guardar")
    self.btn_guardar.setEnabled(False)
else:
    self.combo_categoria.addItems(self._categorias_activas)
self.combo_categoria.setCurrentText(self._categorias_activas[0])
    self.btn_guardar = QPushButton("Guardar")

#Conexión: Al cambiar categoría, recargar la tabla de
calificaciones

self.combo_categoria.currentIndexChanged.connect(self._cargar_datos)

selector_layout.addWidget(lbl_categoria)
selector_layout.addWidget(self.combo_categoria)
main_layout.addLayout(selector_layout)

#2. Tabla de Calificaciones
self.tabla_calificaciones = QTableWidget()
self.tabla_calificaciones.setColumnCount(3)

self.tabla_calificaciones.setHorizontalHeaderLabels(["Matrícula",
"Nombre completo", "Calificación (0-10)"])

self.tabla_calificaciones.horizontalHeader().setSectionResizeMode(QHe
aderView.Stretch)

#Conexión principal: Al editar una celda, guardar
automáticamente

self.tabla_calificaciones.cellChanged.connect(self._guardar_calificac

```



```

ion_celda)

    main_layout.addWidget(self.tabla_calificaciones)

    #Llama a _cargar_datos después de que todos los widgets están
definidos
    if self._categorias_activas:
        self._cargar_datos()
#Método para cargar los datos en la tabla de calificaciones.
    def _cargar_datos(self):

        #Verificar que el combo_categoria no esté vacío antes de
llamar currentText
        categoria_seleccionada = self.combo_categoria.currentText()
        if not categoria_seleccionada or categoria_seleccionada ==
"No hay categorías definidas":
            self.tabla_calificaciones.setRowCount(0)
            return

        #Obtiene los datos desde la logica
        datos =
self.gestor.obtener_alumnos_con_calificaciones(categoria_seleccionada
)

        self.tabla_calificaciones.setRowCount(len(datos))
        self.tabla_calificaciones.blockSignals(True) #Bloquear para
evitar guardado al cargar
        #Llena la tabla con los datos obtenidos
        for fila_indice, alumno_data in enumerate(datos):
            matricula, nombre, valor_db = alumno_data

            #Columna 0 y 1: Datos del alumno (Solo Lectura)
            item_matricula = QTableWidgetItem(matricula)
            item_matricula.setFlags(item_matricula.flags() &
~Qt.ItemIsEditable)
            self.tabla_calificaciones.setItem(fila_indice, 0,
item_matricula)

            #Columna 1: Nombre completo
            item_nombre = QTableWidgetItem(nombre)
            item_nombre.setFlags(item_nombre.flags() &
~Qt.ItemIsEditable)
            self.tabla_calificaciones.setItem(fila_indice, 1,
item_nombre)

            #Columna 2: Calificación

```

```

        valor_str = str(valor_db) if valor_db is not None else ""
        self.tabla_calificaciones.setItem(fila_indice, 2,
QTableWidgetItem(valor_str))

        #desbloquear señales después de cargar los datos
        self.tabla_calificaciones.blockSignals(False)
#Método para guardar la calificación al editar una celda.
    def _guardar_calificacion_celda(self, row, column):
#Si la columna editada no es la de calificaciones, salir
        if column != 2 or not self._categorias_activas:
            return

        matricula = self.tabla_calificaciones.item(row, 0).text()
        categoria = self.combo_categoria.currentText()

        self.tabla_calificaciones.blockSignals(True) #Evitar
recursión
        #Intentar guardar la calificación
        try:
            nuevo_valor_str = self.tabla_calificaciones.item(row,
2).text().strip()

            if not nuevo_valor_str:
                print("Campo vacío. No se registra la calificación.")
                self.tabla_calificaciones.blockSignals(False)
                return

            #Llama al gestor (que ya tiene inicializado
_ponderacion_dao)
            resultado_mensaje =
self.gestor.registrar_calificacion(matricula, categoria,
nuevo_valor_str)

            #Manejo de errores y mensajes
            if "Error" in resultado_mensaje:
                QMessageBox.critical(self, "Error de validación",
resultado_mensaje)

            else:
                print(resultado_mensaje)

        #Captura errores inesperados
        except Exception as e:
            QMessageBox.critical(self, "Error de Entrada", f"Error
inesperado al guardar la nota: {e}")

```

```

        finally:
            self.tabla_calificaciones.blockSignals(False)
#Método para guardar todas las calificaciones manualmente.
    def _guardar_todo_manual(self):

        if self._categorias_activas:
            try:
                self.gestor.recalcular_promedios_grupo()
                QMessageBox.information(self, "Éxito", "Notas
guardadas. El registro es automático por celda y los promedios fueron
recalculados.")
            except AttributeError:
                QMessageBox.critical(self, "Error", "El método
'recalcular_promedios_grupo()' no está implementado en el gestor.")
            except Exception as e:
                QMessageBox.critical(self, "Error", f"Error al
recalcular promedios: {e}")
            else:
                QMessageBox.warning(self, "Advertencia", "No se puede
guardar, no hay categorías definidas.")

```

ventana_registro_profesor.py

```

# presentacion/ventana_registro_profesor.py
from email.mime import application
import sys
from PyQt5.QtWidgets import (
    QDialog, QWidget, QVBoxLayout, QLineEdit,
    QPushButton, QLabel, QMessageBox, QFrame, QHBoxLayout
)
from PyQt5.QtCore import Qt
from PyQt5.QtGui import QFont
from Logica.gestor_autenticacion import GestorAutenticacion

class VentanaRegistroProfesor(QDialog):
    def __init__(self, parent=None):
        super().__init__(parent)
        self.setWindowTitle("DirectAula - Registro de Usuario")
        self.setFixedSize(800, 450)
        self.gestor = GestorAutenticacion()
        self._usuario_registrado = "" # Almacena el nombre de usuario creado
        self._inicializar_ui()

```

```

def _inicializar_ui(self):
    # Estilos de la ventana de registro
    self.setStyleSheet("""
        QDialog {
            background-color: #F7FAFC; /* Gris muy claro */
        }
        QFrame {
            background-color: white;
            border-radius: 15px;
            box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
        }
        QLineEdit {
            border: 1px solid #CBD5E0;
            border-radius: 8px;
            padding: 10px;
            font-size: 14px;
        }
        QLineEdit:focus {
            border: 2px solid #3182CE; /* Azul */
        }
        QPushButton#btnGuardar {
            background-color: #3182CE;
            color: white;
            border: none;
            border-radius: 8px;
            font-size: 16px;
            font-weight: bold;
            height: 45px;
        }
        QPushButton#btnGuardar:hover {
            background-color: #2C5282;
        }
    """)

    main_layout = QVBoxLayout(self)
    main_layout.setAlignment(Qt.AlignCenter)

    main_frame = QFrame()
    main_frame.setFixedSize(350, 500)
    frame_layout = QVBoxLayout(main_frame)
    frame_layout.setAlignment(Qt.AlignTop)
    frame_layout.setSpacing(15)
    frame_layout.setContentsMargins(30, 30, 30, 30)

    # Título
    lbl_titulo = QLabel("Nuevo Profesor")
    lbl_titulo.setAlignment(Qt.AlignCenter)
    lbl_titulo.setFont(QFont("Arial", 16, QFont.Bold))
    lbl_titulo.setStyleSheet("color: #2D3748; margin-bottom: 20px;")
    frame_layout.addWidget(lbl_titulo)

    # Campos de formulario

```

```

self.campo_nombre = QLineEdit()
self.campo_nombre.setPlaceholderText("Nombre Completo")
self.campo_nombre.setFixedHeight(40)
frame_layout.addWidget(self.campo_nombre)

self.campo_email = QLineEdit()
self.campo_email.setPlaceholderText("Correo Electrónico (Opcional)")
self.campo_email.setFixedHeight(40)
frame_layout.addWidget(self.campo_email)

self.campo_usuario = QLineEdit()
self.campo_usuario.setPlaceholderText("Nombre de Usuario (Único)")
self.campo_usuario.setFixedHeight(40)
frame_layout.addWidget(self.campo_usuario)

self.campo_password = QLineEdit()
self.campo_password.setPlaceholderText("Contraseña")
self.campo_password.setEchoMode(QLineEdit.Password)
self.campo_password.setFixedHeight(40)
frame_layout.addWidget(self.campo_password)

self.campo_confirmar_password = QLineEdit()
self.campo_confirmar_password.setPlaceholderText("Confirmar Contraseña")
self.campo_confirmar_password.setEchoMode(QLineEdit.Password)
self.campo_confirmar_password.setFixedHeight(40)
frame_layout.addWidget(self.campo_confirmar_password)

frame_layout.addStretch(1)

# Botón Guardar
btn_guardar = QPushButton("Registrar Profesor")
btn_guardar.setObjectName("btnGuardar")
btn_guardar.clicked.connect(self._registrar_profesor)
frame_layout.addWidget(btn_guardar)

# Botón Cancelar
btn_cancelar = QPushButton("Cancelar")
btn_cancelar.setStyleSheet("border: none; color: #718096; font-size: 13px;
margin-top: 5px;")
btn_cancelar.clicked.connect(self.reject)
frame_layout.addWidget(btn_cancelar)

main_layout.addWidget(main_frame)

# Conectar eventos de teclado
self.campo_confirmar_password.returnPressed.connect(btn_guardar.click)

def _registrar_profesor(self):
    """Intenta registrar un nuevo profesor en la base de datos."""
    nombre = self.campo_nombre.text().strip()
    email = self.campo_email.text().strip()
    usuario = self.campo_usuario.text().strip()

```

```

password = self.campo_password.text()
confirm_password = self.campo_confirmar_password.text()

# 1. Validación de campos obligatorios
if not nombre or not usuario or not password or not confirm_password:
    QMessageBox.warning(self, "Error de Validación", "Todos los campos con (*)
son obligatorios.")
    return

# 2. Validación de contraseñas
if password != confirm_password:
    QMessageBox.warning(self, "Error de Contraseña", "Las contraseñas no
coinciden.")
    self.campo_password.setText("")
    self.campo_confirmar_password.setText("")
    self.campo_password.setFocus()
    return

# 3. Intentar el registro
resultado = self.gestor.registrar_profesor(nombre, usuario, password, email)

if resultado:
    # Si el registro fue exitoso:
    self._usuario_registrado = usuario
    QMessageBox.information(self, "Registro Exitoso", f"Profesor '{nombre}'
registrado. Ahora puedes iniciar sesión.")
    # Indica a la ventana padre que la operación fue aceptada (QDialog.Accepted)
    self.accept()
else:
    QMessageBox.critical(self, "Error de Registro", "El nombre de usuario ya
existe o hubo un error en la base de datos.")
    self.campo_usuario.selectAll()
    self.campo_usuario.setFocus()

def get_usuario_registrado(self):
    """Método público para retornar el usuario registrado."""
    return self._usuario_registrado

if __name__ == "__main__":
    app = application([])
    ventana = VentanaRegistroProfesor()
    if ventana.exec_() == QDialog.Accepted:
        print(f"Usuario creado: {ventana.get_usuario_registrado()}")
    else:
        print("Registro cancelado.")
    sys.exit(0)

```

[app.py](#)

```

#app.py, aqui se inicia la aplicación y se muestra la ventana de
login

import sys #Para manejo de rutas y sistema
import os #Para manejo de rutas
# Importaciones PyQt5 necesarias:
from PyQt5.QtWidgets import (
    QApplication, QMainWindow, QWidget, QVBoxLayout, QPushButton,
    QLabel, QStyleFactory, QDialog, QMessageBox, QFileDialog
)
#Esta línea es necesaria para alinear el título al centro
from PyQt5.QtCore import Qt
# Importaciones de nuestros otros archivos:
from Presentacion.ventana_grupos import VentanaGrupos
from Presentacion.ventana_calificaciones_menu import
VentanaCalificacionesMenu
from Presentacion.ventana_alumnos import VentanaAlumnos
from Presentacion.ventana_asistencia import VentanaAsistencia
from Presentacion.seleccion_grupo import SeleccionGrupo
from Logica.gestor_exportacion import GestorExportaciones
from Presentacion.seleccion_grupo import SeleccionGrupo

#1. IMPORTAR LA VENTANA DE LOGIN
from Presentacion.ventana_login import VentanaLogin
#Clase de la ventana principal del menú
class VentanaMenuPrincipal(QMainWindow):
#Método constructor de la ventana principal
    def __init__(self):
        super().__init__()
        self.setWindowTitle("DirectAula - Sistema de Gestión")
        self.resize(450, 300) #Tamaño inicial de la ventana

        central_widget = QWidget() #Crear un widget central
        self.setCentralWidget(central_widget)
        #Crear un layout vertical para los botones
        layout = QVBoxLayout(central_widget)

        #Título principal
        lbl_titulo = QLabel("DirectAula - Menú Principal")
        lbl_titulo.setObjectName("titulo_principal")
        lbl_titulo.setAlignment(Qt.AlignCenter)
        layout.addWidget(lbl_titulo)

```

```

#1. Botón CU1: Administrar Grupos
btn_grupos = QPushButton("Administrar Grupos")
btn_grupos.clicked.connect(self.abrir_ventana_grupos)
btn_grupos.setObjectName("btn_exportar") #Color azul
layout.addWidget(btn_grupos)

#2. Botón CU2: Administrar Alumnos
btn_alumnos = QPushButton("Administrar Alumnos")
btn_alumnos.clicked.connect(self.abrir_ventana_alumnos)
btn_alumnos.setObjectName("btn_exportar") #Viene de style.css
layout.addWidget(btn_alumnos)

#3. Botón CU4: Registrar Asistencia
btn_asistencia = QPushButton("Registrar Asistencia")
btn_asistencia.clicked.connect(self.abrir_ventana_asistencia)
btn_asistencia.setObjectName("btn_exportar")
layout.addWidget(btn_asistencia)

#4. Botón CU3/CU5: Calificaciones
btn_calificaciones = QPushButton("Registrar Calificaciones")

btn_calificaciones.clicked.connect(self.abrir_ventana_calificaciones)
btn_calificaciones.setObjectName("btn_exportar")
layout.addWidget(btn_calificaciones)

#5. Botón exportar
btn_exportar = QPushButton("Exportar Datos")
btn_exportar.setObjectName("btn_exportar")
btn_exportar.clicked.connect(self.exportar_datos_excel)
layout.addWidget(btn_exportar)

#Métodos para abrir las diferentes ventanas
def abrir_ventana_grupos(self):
    #Ventana de Grupos (CU1)
    self.ventana_grupos = VentanaGrupos()
    self.ventana_grupos.show()

def abrir_ventana_alumnos(self):
    #Ventana de Alumnos (CU2)
    dialogo = SeleccionGrupo("Administrar Alumnos", self)
    if dialogo.exec_() == QDialog.Accepted:
        grupo_id = dialogo.get_grupo_id()
        nombre_grupo = dialogo.combo_grupos.currentText()

```



```

        self.ventana_alumnos = VentanaAlumnos(grupo_id=grupo_id,
nombre_grupo=nombre_grupo)
        self.ventana_alumnos.show()

def abrir_ventana_asistencia(self):
    #Ventana de Asistencia (CU4)
    dialogo = SeleccionGrupo("Registrar Asistencia", self)
    if dialogo.exec_() == QDialog.Accepted:
        grupo_id = dialogo.get_grupo_id()
        nombre_grupo = dialogo.combo_grupos.currentText()
        self.ventana_asistencia =
VentanaAsistencia(grupo_id=grupo_id, nombre_grupo=nombre_grupo)
        self.ventana_asistencia.show()

def abrir_ventana_calificaciones(self):
    #Ventana de Calificaciones (CU3/CU5)
    dialogo = SeleccionGrupo("Gestión de Calificaciones", self)
    if dialogo.exec_() == QDialog.Accepted:
        grupo_id = dialogo.get_grupo_id()
        nombre_grupo = dialogo.combo_grupos.currentText()
        self.ventana_calificaciones =
VentanaCalificacionesMenu(grupo_id=grupo_id,
nombre_grupo=nombre_grupo)
        self.ventana_calificaciones.show()

def exportar_datos_excel(self):
    #Exportar datos de un grupo completo a Excel
    dialogo = SeleccionGrupo("Exportar Datos a Excel", self)
    if dialogo.exec_() == QDialog.Accepted: #Si se presionó
Aceptar

        grupo_id = dialogo.get_grupo_id()
        nombre_grupo = dialogo.combo_grupos.currentText()
        #Abrir un diálogo para seleccionar el directorio donde
guardar el archivo
        directorio = QFileDialog.getExistingDirectory(
            self,
            "Seleccionar carpeta para guardar el archivo Excel",
            "",
            QFileDialog.ShowDirsOnly
        )

        if not directorio:

```

```

        return

        QMessageBox.information(self, "Exportando",
                                "Exportando todos los datos a
Excel.")

        gestor_export = GestorExportaciones() #Crear el gestor de
exportaciones
        exito, mensaje, ruta_archivo =
gestor_export.exportar_grupo_completo(
            grupo_id,
            nombre_grupo,
            directorio
        )

        if exito: #Si la exportación fue exitosa
            QMessageBox.information(self, "Exportación Exitosa",
                                    f"{mensaje}\n\nArchivo guardado
en:\n{ruta_archivo}")
        else:
            QMessageBox.critical(self, "Error en Exportación",
                                mensaje)

#Punto de entrada de la aplicación
if __name__ == '__main__': #Si se ejecuta este archivo directamente
    #Crear la aplicación PyQt5
    QApplication.setStyle(QStyleFactory.create('Fusion'))
    app = QApplication(sys.argv)
    directorio_actual = os.path.dirname(os.path.abspath(__file__))
    ruta_css = os.path.join(directorio_actual, 'style.css')

    try:
        with open(ruta_css, 'r', encoding='utf-8-sig') as f: #Abrir
el archivo CSS con manejo de caracteres especiales
            app.setStyleSheet(f.read())
            print(f"Éxito: style.css cargado desde: {ruta_css}")
    except FileNotFoundError:
        print(f"Advertencia: El archivo style.css no fue encontrado
en la ruta: {ruta_css}")
    except UnicodeDecodeError as e:
        print(f"Error de encoding: {e}. Intenta guardar style.css en
UTF-8 sin BOM.")

```

```
ventana_login = VentanaLogin()
ventana_login.show()
sys.exit(app.exec_())
```

[model.py](#)

```
#Archivo encargado de definir las clases de modelo de datos, como
Alumno, Grupo, Asistencia, Calificación, etc.
from datetime import date #Para manejar fechas en calificaciones y
asistencias
#Clase alumno para representar a un estudiante dentro de un grupo
class Alumno:
    #Método constructor para inicializar un objeto Alumno
    def __init__(self, matricula, nombre_completo, datos_contacto,
email):
        self._matricula = matricula
        self._nombre_completo = nombre_completo
        self._datos_contacto = datos_contacto
        self._email = email
        self._grupo_id = None #Referencia al grupo al que pertenece

# Getters
    def get_matricula(self):
        return self._matricula

    def get_nombre_completo(self):
        return self._nombre_completo

    def get_datos_contacto(self):
        return self._datos_contacto

    def get_email(self):
        return self._email

#Setters
    def set_nombre_completo(self, nombre_completo):
        self._nombre_completo = nombre_completo

    def set_datos_contacto(self, contacto):
        self._datos_contacto = contacto
```

```

def set_email(self, email):
    self._email = email

    # Método para validación interna de datos. Como mínimo, debe
    tener matrícula y nombre.
    def es_valido(self):
        return bool(self._matricula) and bool(self._nombre_completo)
#Clase Asistencia para representar el estado de asistencia de un
alumno en una fecha específica
class Asistencia:
    #Método constructor para inicializar un objeto Asistencia
    def __init__(self, matricula, fecha, estado="Presente"):
        self._matricula = matricula
        self._fecha = fecha
        self._estado = estado

    def get_matricula(self):
        return self._matricula

    def get_fecha(self):
        return self._fecha

    def get_estado(self):
        return self._estado

    def set_estado(self, estado):
        self._estado = estado

#Clase Grupo para representar un grupo o curso académico
class Grupo:
    #Usaremos el id como clave primaria interna, y el nombre/ciclo
    para mostrar en la UI
    def __init__(self, grupo_id, nombre, ciclo_escolar):
        self._grupo_id = grupo_id #Usado internamente para
referencias
        self._nombre = nombre
        self._ciclo_escolar = ciclo_escolar

    def get_id(self):
        return self._grupo_id

```

```

def get_nombre(self):
    return self._nombre

def get_ciclo(self):
    return self._ciclo_escolar
#Clase CategoriaEvaluacion para representar una categoría de
evaluación dentro de un grupo
class CategoriaEvaluacion:
    #Método constructor para inicializar un objeto
    CategoriaEvaluacion
    def __init__(self, grupo_id, nombre_categoria, peso_porcentual,
max_items=1):
        self._grupo_id = grupo_id
        self._nombre_categoria = nombre_categoria
        self._peso_porcentual = peso_porcentual # Ej: 30.0 (30%)
        self._max_items = max_items # BR.9 (No visible pero
importante si lo implementas)

    # Getters(para el DAO)
    def get_grupo_id(self):
        return self._grupo_id

    def get_nombre_categoria(self):
        return self._nombre_categoria

    def get_peso_porcentual(self):
        return self._peso_porcentual

    def get_max_items(self):
        return self._max_items

    # Setters (para modificaciones en la UI)
    def set_peso_porcentual(self, peso):
        self._peso_porcentual = peso

#Clase Calificacion para representar una calificación individual de
un alumno en una categoría en una fecha
class Calificacion:
    #Método constructor para inicializar un objeto Calificacion
    def __init__(self, matricula, categoria, valor, fecha=None):
        self._matricula = matricula
        self._categoria = categoria

```

```

        self._valor = valor
        #Si no se proporciona fecha, usa la de hoy (ISO 8601:
        YYYY-MM-DD)
        self._fecha = fecha if fecha is not None else
        date.today().isoformat()

    #Getters
    def get_matricula(self):
        return self._matricula

    def get_categoria(self):
        return self._categoria

    def get_valor(self):
        return self._valor

    def get_fecha(self):
        return self._fecha

    #Métodos para tuplas de BD que sirven para inserciones y
    actualizaciones
    def to_tuple(self):
        return (self._matricula, self._categoria, self._fecha,
        self._valor)

```

style.css

```

/* Estilo general de la aplicación */
QWidget {
    background-color: #f7f9fc;
    font-family: Arial, sans-serif;
    font-size: 14px;
}

/* Encabezados y títulos */
QLabel {
    color: #333333;
}

/* Campo de Búsqueda (QLineEdit) */

```

```
QLineEdit {
    padding: 5px;
    border: 1px solid #cccccc;
    border-radius: 5px;
    background-color: white;
    color: #000000;
}

/*Campo de búsqueda (QLineEdit) */
QLineEdit:focus {
    border: 2px solid #0C6930;
}

/* Placeholder Text en QLineEdit, placeholder es el texto que
aparece cuando el campo está vacío */
QLineEdit[placeholderText] {
    color: #999999;
}

/* Tabla de alumnos (QTableWidget) */
QTableWidget {
    background-color: white;
    lighting-color: #dddddd;
}

/* Encabezado de la tabla */
QHeaderView::section {
    background-color: #e0e6f0;
    padding: 6px;
    border: 1px solid #cccccc;
    font-weight: bold;
}

/* Estilo general de botones */
QPushButton {
    background-color: #4A36BA;
    color: white;
    border-radius: 5px;
    padding: 6px 12px;
    font-weight: bold;
    border: none;
    font-size: 12px;
    min-height: 28px;
}

/* Efecto hover para botones, hover es cuando el cursor está sobre el
```

```
botón */
QPushButton:hover {
    background-color: #3664BA;
}
/* Efecto presionado para botones */
QPushButton:pressed {
    background-color: #26548A;
}

/* Botón Agregar (Verde) */
#btn_agregar {
    background-color: #0C6930;
}

#btn_agregar:hover {
    background-color: #45AC39;
}

/* Botón Editar (Amarillo) */
#btn_editar {
    background-color: #E3F122;
    color: #333333;
}

#btn_editar:hover {
    background-color: #D3E10E;
}

/* Botón Eliminar (Rojo) */
#btn_eliminar {
    background-color: #E40C0E;
    color: white;
}

#btn_eliminar:hover {
    background-color: #C1312F;
}

/* Botón Exportar (Azul) */
#btn_exportar {
    background-color: #4A36BA;
    color: white;
}
```



```
}

#btn_exportar:hover {
    background-color: #3664BA;
}

/* Estilo para los Títulos Principales */
#titulo_principal {
    color: #2A0E90;
    font-size: 24px;
    font-weight: bold;
    padding-bottom: 10px;
}

/* Estilo para los Subtítulos y Secciones */
.subtitulo {
    color: #2E0F9E;
    font-size: 16px;
    font-weight: bold;
}
```